Index

Sr. No	Title	Date	Page No.	Sign
1.	Design and implement algorithms to encrypt and decrypt messages using classical substitution and transposition techniques.		1 - 5	
2.	Implement the RSA algorithm for public-key encryption and decryption, and explore its properties and security considerations.		6 - 9	
3.	Implement algorithms to generate and verify message authentication codes (MACs) for ensuring data integrity and authenticity.		10 - 12	
4.	Implement digital signature algorithms such as RSA-based signatures, and verify the integrity and authenticity of digitally signed messages.		13 - 16	
5.	Implement the Diffie-Hellman key exchange algorithm to securely exchange keys between two entities over an insecure network.		17 - 22	
6.	Configure IPsec on network devices to provide secure communication and protect against unauthorized access and attacks.		23 - 34	
7.	Configure and implement secure web communication using SSL/TLS protocols, including certificate management and secure session establishment.		35 – 42	
8.	Set up and configure an intrusion detection system (IDS) to monitor network traffic and detect potential security breaches or malicious activities.		43 – 47	
9.	Analyze and identify malware samples using antivirus tools, analyze their behavior, and develop countermeasures to mitigate their impact.		48 – 55	
10.	Configure and test firewall rules to control network traffic, filter packets based on specified criteria, and protect network resources from unauthorized access.		56 – 63	

Practical No. 1

Aim: Design and implement algorithms to encrypt and decrypt messages using classical substitution and transposition techniques.

Theory:

Substitution Cipher

A **substitution cipher** encrypts a message by replacing each letter (or group of letters) with another symbol, while keeping the letters in their original position.

Main types

- **Monoalphabetic**: one mapping for the whole message.
  - *Caesar cipher*: shift every letter by a fixed number. With shift 3: HELLO → KHOOR.
  - *Keyword/random alphabet*: the cipher alphabet is a shuffled version of A-Z.
- **Polyalphabetic**: several mappings, switched in turn.
  - *Vigenère cipher*: a repeating keyword decides the shift for each letter. Key KEY shifts by 10, 4, 24, then repeats.
- **Polygraphic**: replaces groups of letters.
  - *Playfair cipher*: encrypts pairs of letters using a 5×5 key grid.
- **One-time pad**: a truly random key as long as the message. It is theoretically unbreakable if used correctly.

Transposition Cipher

A **transposition cipher** encrypts a message by rearranging the positions of the letters, while keeping the letters themselves unchanged.

Main types

- **Rail fence cipher**: write the text in a zigzag over N rows, then read row by row.
  - HELLOWORLD on 2 rails: row 1 = HLOOL, row 2 = ELWRD → ciphertext HLOOLELWRD.
- **Columnar transposition**: write the text in rows under a keyword, then read the columns in the order of the keyword's alphabetical ranking.
  - Key ZEBRA gives column order by letters: A, B, E, R, Z.
- **Double transposition**: apply columnar transposition twice with different keys. This is much stronger and was used in real wartime systems.

Code:

Substitution Cipher

```
def caesar_encrypt(text, key):
  result = ""

  for char in text:
    if 'A' <= char <= 'Z':
      result += chr((ord(char) - ord('A') + key) % 26 + ord('A'))
    elif 'a' <= char <= 'z':
      result += chr((ord(char) - ord('a') + key) % 26 + ord('a'))
    elif '0' <= char <= '9':
      result += chr((ord(char) - ord('0') + key) % 10 + ord('0'))
    else:
      result += char
  return result

def caesar_decrypt(ciphertext, key):
  result = ""
  for char in ciphertext:
    if 'A' <= char <= 'Z':
      result += chr((ord(char) - ord('A') - key) % 26 + ord('A'))
    elif 'a' <= char <= 'z':
      result += chr((ord(char) - ord('a') - key) % 26 + ord('a'))
    elif '0' <= char <= '9':
      result += chr((ord(char) - ord('0') - key) % 10 + ord('0'))
    else:
      result += char
  return result

plaintext = input("Enter plaintext: ")

while True:
  try:
    en_key = int(input("Enter Caesar Cipher key: "))
    break
  except ValueError:
    print("Error: Key must be an integer.")

encrypted = caesar_encrypt(plaintext, en_key)
print("Encryption was Successful : ")

de_key = int(input("Enter key for Decryption : "))

print("\nPlaintext :", plaintext)
print("Encrypted message :", encrypted)
```

```
if de_key == en_key:
   decrypted = caesar_decrypt(encrypted, de_key)
   print("Decrypted message :", decrypted)
else:
   print("Wrong Key entered therefore the Decryption Failed.")
```

Transposition Cipher

```
import math

def get_key_order(key):
   """Return column order based on alphabetical order of key letters."""
   key_list = list(key)
   order = sorted(range(len(key_list)), key=lambda k: key_list[k])
   return order


def columnar_encrypt(plaintext, key):
   plaintext = plaintext.replace(" ", "")
   col = len(key)
   row = math.ceil(len(plaintext) / col)

   padded_text = plaintext.ljust(row * col, 'X')

   grid = [padded_text[i:i + col] for i in range(0, len(padded_text), col)]

   order = get_key_order(key)

   ciphertext = ""
   for col_index in order:
      for r in range(row):
         ciphertext += grid[r][col_index]

   return ciphertext


def columnar_decrypt(ciphertext, key):
   col = len(key)
   row = math.ceil(len(ciphertext) / col)
   order = get_key_order(key)

   grid = [['' for _ in range(col)] for _ in range(row)]

   index = 0
```

```
    for col_index in order:
        for r in range(row):
            grid[r][col_index] = ciphertext[index]
            index += 1

    plaintext = ""
    for r in range(row):
        plaintext += "".join(grid[r])

    return plaintext.rstrip('X')

plaintext = input("Enter plaintext: ")
En_key = input("Enter keyword: ")

cipher = columnar_encrypt(plaintext, En_key)
print("\nEncryption is Successful")

De_key = input("Enter keyword for the Decryption: ")
if De_key == En_key:
    decrypted = columnar_decrypt(cipher, De_key)
else:
    decrypted = "Invalid keyword, therefore Text cannot be decrypted"

print("\nOriginal Text  :", plaintext)
print("Encrypted Text :", cipher)
print("Decrypted Text :", decrypted)
```

Output:

Substitution Cipher

```
Enter plaintext: Hello
Enter Caesar Cipher key: 2
Encryption was Successful :
Enter key for Decryption : 2

Plaintext : Hello
Encrypted message : Jgnnq
Decrypted message : Hello
```

Transposition Cipher

```
Enter plaintext: Hello
Enter keyword: World

Encryption is Successful
Enter keyword for the Decryption: World

Original Text  : Hello
Encrypted Text : Holel
Decrypted Text : Hello
```

Practical No. 2

Aim: Implement the RSA algorithm for public-key encryption and decryption, and explore its properties and security considerations.

Theory:

RSA Algorithm

RSA (Rivest-Shamir-Adleman) is an **asymmetric (public-key) cryptosystem**. It uses two linked keys: a **public key** that anyone can use to encrypt, and a **private key** that only the owner uses to decrypt. Its security rests on the fact that multiplying two large primes is easy, but factoring the result back into those primes is extremely hard.

Core Idea

- Anyone can lock a message with your public key.
- Only you, holding the private key, can unlock it.
- No secret key needs to be shared in advance, which solves the key-distribution problem of symmetric ciphers.

Steps of the Algorithm

1. Key generation

1. Choose two large distinct primes, **p** and **q**.
2. Compute **n = p × q**. This is the modulus and is part of both keys.
3. Compute **ϕ(n) = (p − 1)(q − 1)**, Euler's totient.
4. Choose a public exponent **e** such that 1 < e < ϕ(n) and gcd(e, ϕ(n)) = 1. A common choice is 65537.
5. Compute the private exponent **d** such that **d × e ≡ 1 (mod ϕ(n))**. This is the modular inverse of e.

- **Public key:** (e, n)
- **Private key:** (d, n)
- p, q and ϕ(n) are kept secret or discarded.

2. Encryption

- Convert the message to a number **m**, where m < n.
- Ciphertext: $\mathbf{c = m^e \bmod n}$

3. Decryption

- Plaintext: $\mathbf{m = c^d \bmod n}$

This works because of Euler's theorem: m^(e·d) ≡ m (mod n).

Code:

```
def is_prime(num):
    if num < 2:
        return False
    for i in range(2, int(num ** 0.5) + 1):
        if num % i == 0:
            return False
    return True

def gcd(a, b):
    while b != 0:
        a, b = b, a % b
    return a

def mod_inverse(e, phi):
    for d in range(2, phi):
        if (d * e) % phi == 1:
            return d
    return None

print("=== RSA Key Generation ===")

p = int(input("Enter 1st prime number (p): "))
q = int(input("Enter 2nd prime number (q): "))

if not (is_prime(p) and is_prime(q)):
    print("Both numbers must be prime!")
    exit()

n = p * q
phi = (p - 1) * (q - 1)

e = 2
while e < phi:
    if gcd(e, phi) == 1:
        break
    e += 1

d = mod_inverse(e, phi)

print("\nModulus (n)      :", n)
print("Phi(n)           :", phi)
print("Public Key  (e, n):", (e, n))
print("Private Key (d, n):", (d, n))

message = input("\nEnter message to encrypt: ")
```

```
print("\nPublic key ",e,",",n," will be used fot Encryption :")
cipher = []
for char in message:
    m = ord(char)
    c = pow(m, e, n)
    cipher.append(c)

print("\n=== Encrypted Message (Ciphertext) ===")
print(cipher)

print("\nPublic key ",d,",",n," will be used fot Decryption :")
decrypted = ""
for c in cipher:
    m = pow(c, d, n)
    decrypted += chr(m)

print("\n=== Decrypted Message ===")
print(decrypted)

if decrypted == message:
    print("\nResult: Decryption Successful! Message matches original.")
else:
    print("\nResult: Decryption Failed!")
```

Output:

```
=== RSA Key Generation ===
Enter 1st prime number (p): 53
Enter 2nd prime number (q): 61

Modulus (n)       : 3233
Phi(n)            : 3120
Public Key  (e, n): (7, 3233)
Private Key (d, n): (1783, 3233)

Enter message to encrypt: hi

Public key  7 , 3233  will be used fot Encryption :

=== Encrypted Message (Ciphertext) ===
[3052, 3020]

Public key  1783 , 3233  will be used fot Decryption :

=== Decrypted Message ===
hi

Result: Decryption Successful! Message matches original.
```

Practical No. 3

Aim: Implement algorithms to generate and verify message authentication codes (MACs) for ensuring data integrity and authenticity.

Theory:

Message Authentication Code (MAC)

A Message Authentication Code (MAC) is a short fixed-size tag computed from a message and a secret key shared by the sender and receiver. The receiver recomputes the tag and compares it. If the tags match, the message was not changed (integrity) and it came from someone who knows the key (authenticity).

Key Points

1. **How it works**
    - Sender computes tag = MAC(key, message) and sends message + tag.
    - Receiver computes MAC(key, received_message) with the same key.
    - If the two tags are equal, accept the message. If not, reject it.
2. **Why it gives integrity:** Changing even one bit of the message produces a completely different tag. An attacker who alters the message cannot fix the tag without the key.
3. **Why it gives authenticity:** Only parties holding the secret key can produce a valid tag, so a valid tag shows the message came from the key holder.
4. **MAC vs. plain hash:** A plain hash like SHA-256 has no key, so an attacker can change the message and simply recompute the hash. A MAC needs the key, so it cannot be forged.
5. **MAC vs. digital signature**
    - MAC uses one shared symmetric key and is very fast.
    - A digital signature uses a private/public key pair.
    - A MAC cannot provide non-repudiation: since both sides hold the key, either could have made the tag, so the sender can deny sending it. Signatures do not have this problem.
6. **Common MAC types**
    - **HMAC** is built from a hash function: HMAC(K, m) = H((K $\oplus$ opad) || H((K $\oplus$ ipad) || m)). It is the most widely used, for example HMAC-SHA256.
    - **CMAC** is built from a block cipher such as AES.
    - **Poly1305 and GMAC** are fast MACs used in authenticated encryption (ChaCha20-Poly1305, AES-GCM).

Code:

```
import hmac
import hashlib

def generate_mac(message, secret_key):
    """
    Generate an HMAC (using SHA-256) for the given message,
    using the shared secret key.
    """
    message_bytes = message.encode()
    key_bytes = secret_key.encode()

    mac = hmac.new(key_bytes, message_bytes, hashlib.sha256)
    return mac.hexdigest()  # return MAC as a readable hex string

def verify_mac(message, received_mac, secret_key):
    """
    Recompute the MAC for the received message using the same key,
    then compare it with the MAC that was sent.
    If they match -> message is authentic and unmodified.
    If they don't  -> message was tampered with OR key is wrong.
    """
    expected_mac = generate_mac(message, secret_key)

    return hmac.compare_digest(expected_mac, received_mac)

print("===== MAC (HMAC-SHA256) DEMO =====\n")

secret_key = input("Enter the Secret key : ")
message = input("Enter the message to send: ")

mac = generate_mac(message, secret_key)
print(f"\n[Sender] Message  : {message}")
print(f"[Sender] MAC sent : {mac}")

print("\n--- Message travels over the network ---\n")

tamper = input("Simulate tampering with the message? (y/n): ").strip().lower()
received_message = message
if tamper == 'y':
    t_message = input("Enter the keywords to tamper the message: ")
    received_message = message + " " + t_message

received_mac = mac

print(f"[Receiver] Message received : {received_message}")
```

```
is_valid = verify_mac(received_message, received_mac, secret_key)

if is_valid:
    print("[Receiver] Result: VALID - message is authentic and unaltered.")
else:
    print("[Receiver] Result: INVALID - message was tampered with or key mismatch!")
```

Output:

```
===== MAC (HMAC-SHA256) DEMO =====

Enter the Secret key : World
Enter the message to send: Hello

[Sender] Message  : Hello
[Sender] MAC sent : 46288437613044114d21e7fad79837c12336202f4c85008548fb226693426f56

--- Message travels over the network ---

Simulate tampering with the message? (y/n): Y
Enter the keywords to tamper the message: World
[Receiver] Message received : Hello World
[Receiver] Result: INVALID - message was tampered with or key mismatch!
```

Practical No. 4

Aim: Implement digital signature algorithms such as RSA-based signatures, and verify the integrity and authenticity of digitally signed messages.

Theory:

Digital Signature Algorithms

A digital signature algorithm lets a sender sign data with a **private key**, and anyone can check it with the matching **public key**. It gives integrity, authenticity, and non-repudiation (the signer can't deny signing), which a MAC cannot give.

How it works

1. Hash the message (for example SHA-256).
2. Sign the hash with the private key. This produces the signature.
3. Send the message and signature.
4. The receiver hashes the message and verifies the signature with the public key. If it verifies, the message is unchanged and came from the key owner.

Main Algorithms

Algorithm	**Based on**	**Typical key size**	**Notes**
RSA (PKCS#1 v1.5, RSA-PSS)	Integer factoring	2048-3072 bits	Widely supported. PSS is the preferred, safer padding. Large keys and signatures.
DSA	Discrete logarithm	2048-3072 bits	Old, now deprecated for new signatures.
ECDSA	Elliptic curves	256 bits (P-256)	Small and fast. Used in TLS, Bitcoin, JWT (ES256). Needs a good random nonce per signature.
EdDSA (Ed25519)	Edwards curves	256 bits	Fast, deterministic (no nonce risk), easy to use safely. Used in SSH, modern TLS.
Schnorr	Discrete logarithm	256 bits	Simple and supports signature aggregation. Used in Bitcoin Taproot.

Code:

```
import random
import hashlib
from math import gcd

def is_prime(num):
    if num < 2:
        return False
    for i in range(2, int(num ** 0.5) + 1):
        if num % i == 0:
            return False
    return True


def mod_inverse(e, phi):
    old_r, r = e, phi
    old_s, s = 1, 0
    while r != 0:
        quotient = old_r // r
        old_r, r = r, old_r - quotient * r
        old_s, s = s, old_s - quotient * s
    return old_s % phi


def generate_keys(p, q):
    if not (is_prime(p) and is_prime(q)):
        raise ValueError("Both p and q must be prime numbers!")
    if p == q:
        raise ValueError("p and q must be distinct primes!")

    n = p * q
    phi = (p - 1) * (q - 1)

    e = random.randrange(2, phi)
    while gcd(e, phi) != 1:
        e = random.randrange(2, phi)

    d = mod_inverse(e, phi)

    return (e, n), (d, n)

def hash_message(message):
    """
    Convert the message into a fixed-size numeric hash using SHA-256,
    then reduce it so it fits within our (small demo) RSA modulus 'n'.
```

```
    """
    digest = hashlib.sha256(message.encode()).hexdigest()
    return int(digest, 16)

def sign_message(message, private_key):
    d, n = private_key
    hashed = hash_message(message) % n
    signature = pow(hashed, d, n)
    return signature

def verify_signature(message, signature, public_key):
    e, n = public_key
    hashed = hash_message(message) % n

    decrypted_hash = pow(signature, e, n)

    return decrypted_hash == hashed

print("===== RSA DIGITAL SIGNATURE DEMO =====\n")

p = int(input("Enter 1st Prime Numbers : "))
q = int(input("Enter 2nd Prime Numbers : "))
public_key, private_key = generate_keys(p, q)

print(f"Public Key  (e, n): {public_key}")
print(f"Private Key (d, n): {private_key}\n")

message = input("Enter the message to sign: ")

signature = sign_message(message, private_key)
print(f"\n[Sender] Message   : {message}")
print(f"[Sender] Signature : {signature}")

print("\n--- Message + Signature sent to receiver ---\n")

tamper = input("Simulate tampering with the message? (y/n): ").strip().lower()
received_message = message
if tamper == 'y':
    tamp_text = input("Please enter keywords to tamper the message :" )
    received_message = message + tamp_text

is_valid = verify_signature(received_message, signature, public_key)

print(f"[Receiver] Message received : {received_message}")
if is_valid:
    print("[Receiver] Result: VALID - signature matches, message is authentic and
unaltered.")
```

else:
  print("[Receiver] Result: INVALID - message was tampered with or signature does not match!")

Output:

```
===== RSA DIGITAL SIGNATURE DEMO =====

Enter 1st Prime Numbers : 53
Enter 2nd Prime Numbers : 61
Public Key  (e, n): (1073, 3233)
Private Key (d, n): (977, 3233)

Enter the message to sign: Hello

[Sender] Message   : Hello
[Sender] Signature : 2018

--- Message + Signature sent to receiver ---

Simulate tampering with the message? (y/n): Y
Please enter keywords to tamper the message : World
[Receiver] Message received : Hello World
[Receiver] Result: INVALID - message was tampered with or signature does not match!
```

Practical No. 5

Aim: Implement the Diffie-Hellman key exchange algorithm to securely exchange keys between two entities over an insecure network.

Theory:

Diffie-Hellman key exchange algorithm

Diffie-Hellman (DH) is a key exchange algorithm that lets two parties create a shared secret key over an insecure channel, without ever sending the key itself. Its security rests on the discrete logarithm problem: given g^a mod p, it is computationally infeasible to find a.

Key Points

1. **Public values:** Both parties agree on a large prime p and a generator g. These are not secret.
2. **Steps**
    - Alice picks a secret a and sends A = g^a mod p.
    - Bob picks a secret b and sends B = g^b mod p.
    - Alice computes K = B^a mod p.
    - Bob computes K = A^b mod p.
    - Both get the same key, because (g^b)^a = (g^a)^b = g^(ab) mod p.
3. **Small example** (p = 23, g = 5)
    - Alice: a = 6, so A = 5^6 mod 23 = 8
    - Bob: b = 15, so B = 5^15 mod 23 = 19
    - Alice: 19^6 mod 23 = 2
    - Bob: 8^15 mod 23 = 2
    - Shared secret = 2. An eavesdropper sees only p, g, 8 and 19.
4. **Why it is secure:** An attacker who sees g, p, A and B would need to solve the discrete logarithm to get a or b. With 2048-bit or larger primes this is not feasible.
5. **Main weakness: no authentication.** DH does not prove who you are talking to, so it is open to man-in-the-middle attacks. An attacker can run one exchange with Alice and another with Bob. The fix is to authenticate the exchange, for example with digital signatures or certificates (as TLS does).
6. **Variants**
    - **ECDH:** the same idea on elliptic curves. Keys are much smaller (256 bits is enough) and it is faster. X25519 is the common choice.
    - **Ephemeral DH (DHE/ECDHE):** a fresh key pair for every session, which gives forward secrecy. If a long-term key leaks later, past sessions stay safe.

Code:

```
def is_prime(n):

  if n <= 1:
    return False

  if n == 2:
    return True

  if n % 2 == 0:
    return False

  i = 3

  while i * i <= n:
    if n % i == 0:
      return False
    i += 2

  return True

def is_primitive_root(g, p):

  if g <= 1 or g >= p:
    return False
  values = set()
  for i in range(1, p):

    value = pow(g, i, p)
    values.add(value)
  return len(values) == p - 1

print("=" * 55)
print("     DIFFIE-HELLMAN KEY EXCHANGE")
print("=" * 55)

while True:

  try:
    p = int(input("\nEnter a prime number (p): "))

    if p <= 2:
      print("Error: p must be a prime number greater than 2.")
      continue

    if is_prime(p):
```

```
            print("Verification: p is a PRIME number.")
            break
        else:
            print("Verification: p is NOT a prime number.")
            print("Please enter a prime number.")

    except ValueError:
        print("Invalid input. Please enter an integer.")

while True:

    try:
        g = int(input("\nEnter the generator/primitive root (g): "))

        if is_primitive_root(g, p):
            print("Verification: g is a PRIMITIVE ROOT modulo p.")
            break
        else:
            print("Verification: g is NOT a primitive root modulo p.")
            print("Please enter a valid primitive root.")

    except ValueError:
        print("Invalid input. Please enter an integer.")

while True:

    try:
        alice_private = int(input("\nEnter Alice's private key: "))

        if alice_private <= 0:
            print("Private key must be greater than 0.")
        else:
            break

    except ValueError:
        print("Invalid input. Please enter an integer.")

while True:

    try:
        bob_private = int(input("Enter Bob's private key: "))

        if bob_private <= 0:
            print("Private key must be greater than 0.")
        else:
            break
```

```
    except ValueError:
        print("Invalid input. Please enter an integer.")

alice_public = pow(g, alice_private, p)

bob_public = pow(g, bob_private, p)

print("\n" + "=" * 55)
print("          KEY GENERATION")
print("=" * 55)

print("Public Prime (p):", p)
print("Public Generator (g):", g)

print("\nAlice:")
print("Private Key:", alice_private)
print("Public Key :", alice_public)

print("\nBob:")
print("Private Key:", bob_private)
print("Public Key :", bob_public)

print("\n" + "=" * 55)
print("          PUBLIC KEY EXCHANGE")
print("=" * 55)

print("Alice sends her public key to Bob:", alice_public)
print("Bob sends his public key to Alice:", bob_public)

alice_shared = pow(bob_public, alice_private, p)

bob_shared = pow(alice_public, bob_private, p)

print("\n" + "=" * 55)
print("          SHARED SECRET")
print("=" * 55)

print("Alice's Shared Secret:", alice_shared)
print("Bob's Shared Secret  :", bob_shared)

print("\n" + "=" * 55)

if alice_shared == bob_shared:

    print("KEY EXCHANGE SUCCESSFUL!")
    print("Both Alice and Bob generated the same shared secret.")
```

```
else:

    print("KEY EXCHANGE FAILED!")
    print("The shared secrets are different.")

print("=" * 55)
```

Output:

```
============================================================
       DIFFIE-HELLMAN KEY EXCHANGE
============================================================

Enter a prime number (p): 61
Verification: p is a PRIME number.

Enter the generator/primitive root (g): 10
Verification: g is a PRIMITIVE ROOT modulo p.

Enter Alice's private key: 15
Enter Bob's private key: 25

============================================================
             KEY GENERATION
============================================================
Public Prime (p): 61
Public Generator (g): 10

Alice:
Private Key: 15
Public Key : 50
```

```
Bob:
Private Key: 25
Public Key : 29

============================================================
             PUBLIC KEY EXCHANGE
============================================================
Alice sends her public key to Bob: 50
Bob sends his public key to Alice: 29

============================================================
             SHARED SECRET
============================================================
Alice's Shared Secret: 50
Bob's Shared Secret  : 50

============================================================
KEY EXCHANGE SUCCESSFUL!
Both Alice and Bob generated the same shared secret.
============================================================
```

Practical No. 6

Aim: Configure IPsec on network devices to provide secure communication and protect against unauthorized access and attacks.

Theory:

IP Security (IPSec)

IP Security (IPsec) is a set of network security protocols used to protect data transmitted over an IP network, such as the Internet. It provides security by authenticating, encrypting, and ensuring the integrity of IP packets during communication between devices.

Features

- **Confidentiality:** Encrypts the data so that unauthorized users cannot read the information.
- **Key Management:** Manages the creation, exchange, and updating of cryptographic keys securely.
- **Tunneling:** Allows IP packets to be encapsulated inside another packet to create a secure communication path.
- **Flexibility:** Can be used in different network types such as point-to-point, site-to-site, and remote access connections.
- **Interoperability:** Works across different devices and vendors because it is an open standard protocol.

Protocols Used in IPSec

- **Encapsulating Security Payload (ESP):** It provides data integrity, encryption, authentication, and anti-replay. It also provides authentication for the payload.
- **Authentication Header (AH):** Provides authentication and data integrity for transmitted data, but does not provide encryption or confidentiality.
- **Internet Key Exchange (IKE):** Establishes Security Association (SA) and exchanges cryptographic keys required for secure communication.

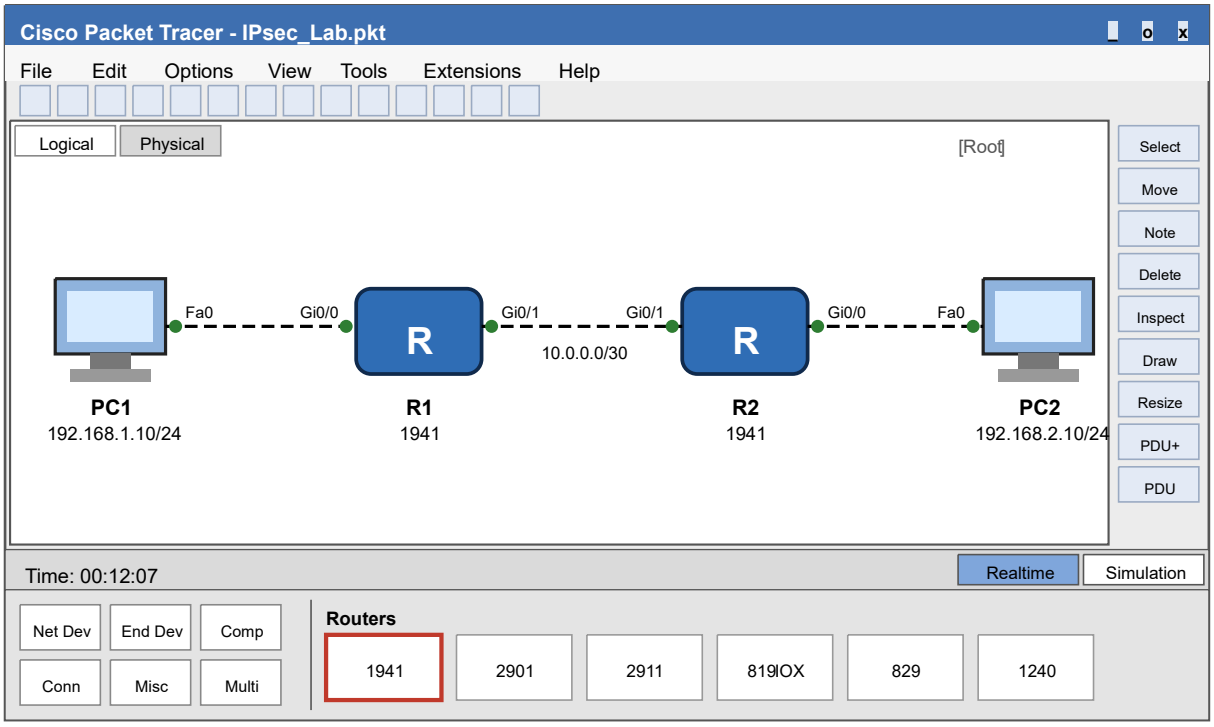
Procedure:

Step 1: Place the devices

1. Open Cisco Packet Tracer and start a new file (File > New).
2. Bottom-left panel: End Devices > drag two PC icons to the workspace.
3. Network Devices > Routers > drag two 1941 routers to the workspace.
4. Arrange left to right: PC1, R1, R2, PC2 and rename them by clicking the device name.

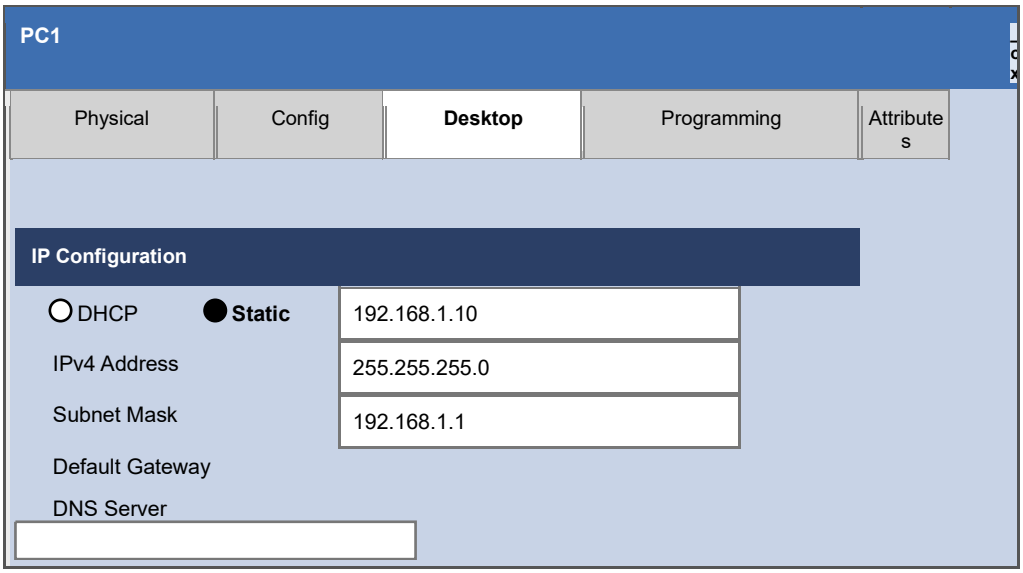
Step 2: Connect the cables

1. Click the Connections icon (lightning bolt) and choose Copper Cross-Over.
2. PC1 FastEthernet0 to R1 GigabitEthernet0/0.
3. R1 GigabitEthernet0/1 to R2 GigabitEthernet0/1.
4. R2 GigabitEthernet0/0 to PC2 FastEthernet0.
5. Link lights stay red until the router interfaces are turned on in Step 3.



Step 3: Configure IP addresses

Input (PCs): click PC1 > Desktop > IP Configuration > select Static and type the values. Repeat for PC2.



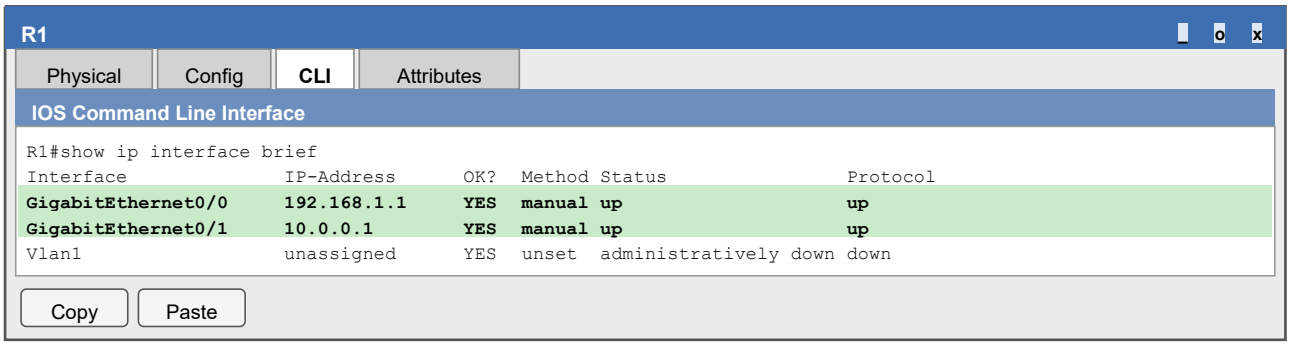
Input (R1): click R1 > **CLI**. If asked 'Continue with configuration dialog?' type `no`.

```
Router> enable
Router# configure terminal
Router(config)# hostname R1
R1(config)# interface gigabitEthernet 0/0
R1(config-if)# ip address 192.168.1.1 255.255.255.0
R1(config-if)# no shutdown
R1(config-if)# exit
R1(config)# interface gigabitEthernet 0/1
R1(config-if)# ip address 10.0.0.1 255.255.255.252
R1(config-if)# no shutdown
R1(config-if)# exit
R1(config)# ip route 192.168.2.0 255.255.255.0 10.0.0.2
R1(config)# end
R1# copy running-config startup-config
```

Input (R2): click R2 > CLI:

```
Router> enable
Router# configure terminal
Router(config)# hostname R2
R2(config)# interface gigabitEthernet 0/0 R2(config-if)# ip address 192.168.2.1 255.255.255.0
R2(config-if)# no shutdown
R2(config-if)# exit
R2(config)# interface gigabitEthernet 0/1 R2(config-if)# ip address 10.0.0.2 255.255.255.252
R2(config-if)# no shutdown
R2(config-if)# exit
R2(config)# ip route 192.168.1.0 255.255.255.0 10.0.0.1
R2(config)# end
R2# copy running-config startup-config
```

Input (check): on R1 type `show ip interface brief`. **Output:**

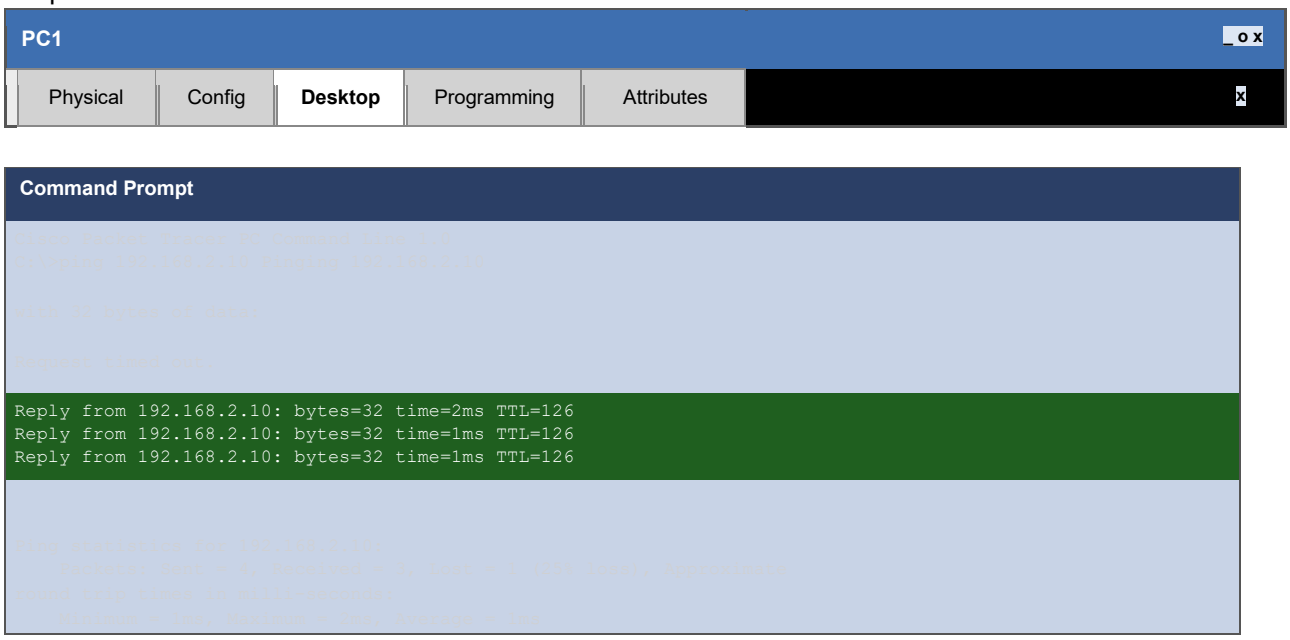


Step 4: Baseline test (before IPsec)

Input: PC1 > **Desktop** > **Command Prompt**:

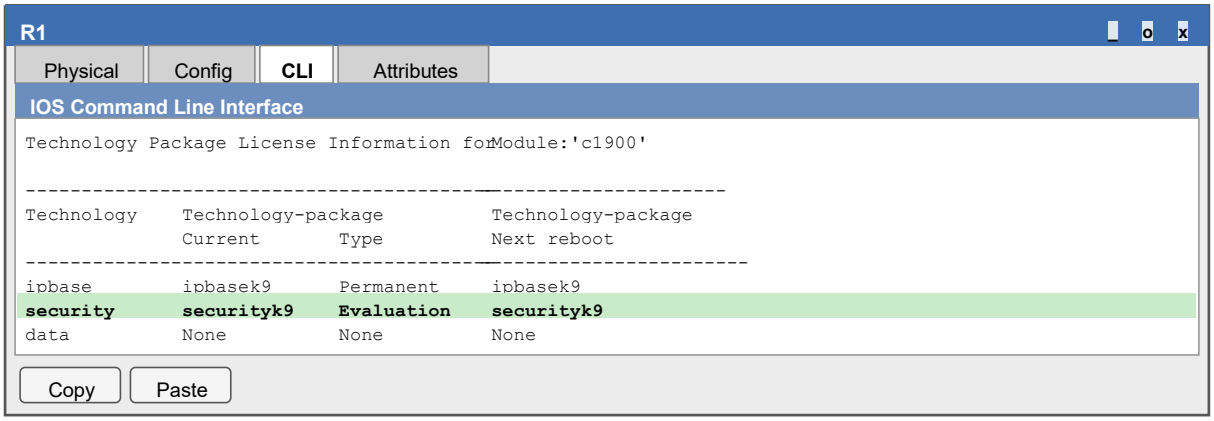
```
C:\> ping 192.168.2.10
```

Output: replies are received (the first request often times out while ARP resolves). TTL = 126 shows the packet crossed two routers.



Step 5: Check and enable the security licence

Input (R1): show version. Look at the Technology Package License Information table at the end.



If securityk9 is not active, input this on both routers, accept the licence (type yes), then reload:

```
R1# configure terminal
R1(config)# license boot module c1900 technology-package securityk9
R1(config)# end
R1# copy running-config startup-config
R1# reload
```

Step 6: Configure IPsec on Router R1

Input: type each block in the R1 CLI.

6.1 Crypto ACL (interesting traffic: LAN 1 to LAN 2; wildcard mask)

```
R1# configure terminal
R1(config)# access-list 110 permit ip 192.168.1.0 0.0.0.255 192.168.2.0 0.0.0.255
```

6.2 IKE Phase 1: ISAKMP policy

```
R1(config)# crypto isakmp policy 10
R1(config-isakmp)# encryption aes 256
R1(config-isakmp)# hash sha
R1(config-isakmp)# authentication pre-share
R1(config-isakmp)# group 5
R1(config-isakmp)# lifetime 86400
R1(config-isakmp)# exit
```

6.3 Pre-shared key (linked to R2's address)

```
R1(config)# crypto isakmp key Lab@IPsec2026 address 10.0.0.2
```

6.4 IKE Phase 2: transform set

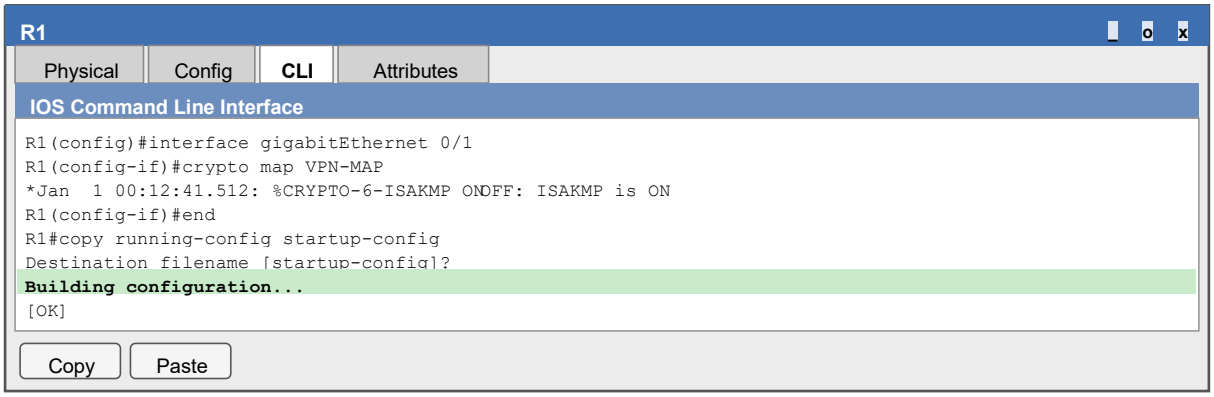
```
R1(config)# crypto ipsec transform-set VPN-TS esp-aes 256 esp-sha-hmac
R1(cfg-crypto-trans)# exit
```

6.5 Crypto map

```
R1(config)# crypto map VPN-MAP 10 ipsec-isakmp
R1(config-crypto-map)# set peer 10.0.0.2
R1(config-crypto-map)# set transform-set VPN-TS
R1(config-crypto-map)# match address 110
R1(config-crypto-map)# exit
```

6.6 Apply the crypto map to the outgoing interface

```
R1(config)# interface gigabitEthernet 0/1 R1(config-if)# crypto map VPN-MAP
R1(config-if)# end
R1# copy running-config startup-config
```



Step 7: Configure IPsec on Router R2 (mirror image)

Rules for R2: ACL is **reversed**, the peer is **R1 (10.0.0.1)**, everything else is **identical**.

Input:

```
R2# configure terminal
R2(config)# access-list 110 permit ip 192.168.2.0 0.0.0.255 192.168.1.0 0.0.0.255

R2(config)# crypto isakmp policy 10
R2(config-isakmp)# encryption aes 256
R2(config-isakmp)# hash sha
R2(config-isakmp)# authentication pre-share
R2(config-isakmp)# group 5
R2(config-isakmp)# lifetime 86400
R2(config-isakmp)# exit

R2(config)# crypto isakmp key Lab@IPsec2026 address 10.0.0.1

R2(config)# crypto ipsec transform-set VPN-TS esp-aes 256 esp-sha-hmac
R2(cfg-crypto-trans)# exit

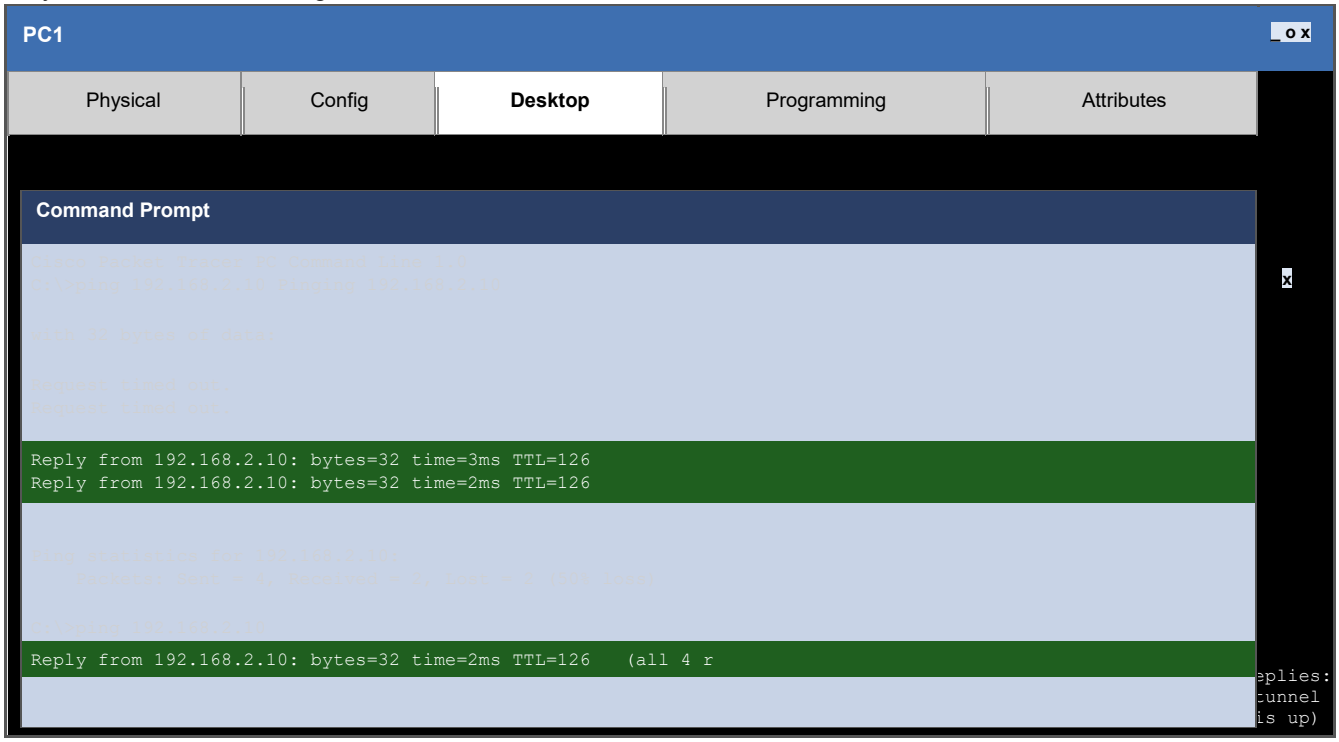
R2(config)# crypto map VPN-MAP 10 ipsec-isakmp
R2(config-crypto-map)# set peer 10.0.0.1
R2(config-crypto-map)# set transform-set VPN-TS
R2(config-crypto-map)# match address 110
R2(config-crypto-map)# exit

R2(config)# interface gigabitEthernet 0/1
R2(config-if)# crypto map VPN-MAP
R2(config-if)# end
R2# copy running-config startup-config
```

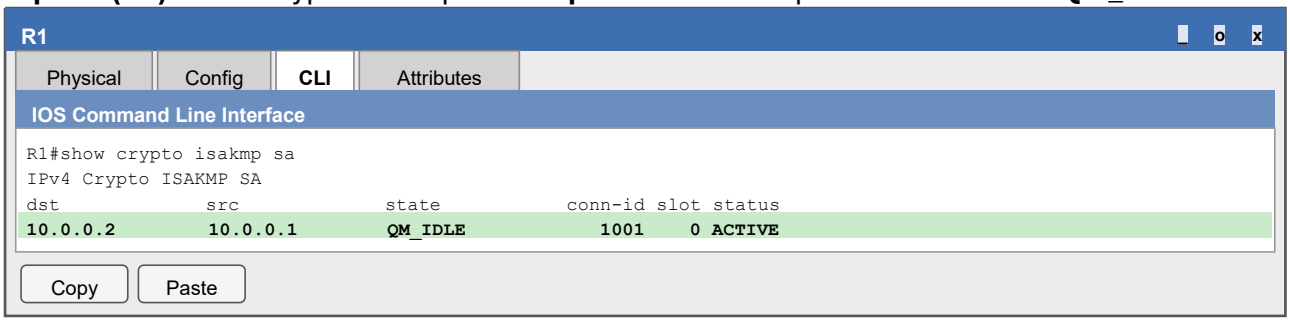
If a command is rejected (for example 'aes 256' or 'group 5'), type **?** to list what your Packet Tracer version supports (such as aes, 3des, group 2) and use the **same** choice on both routers.

Step 8: Verify the IPsec tunnel

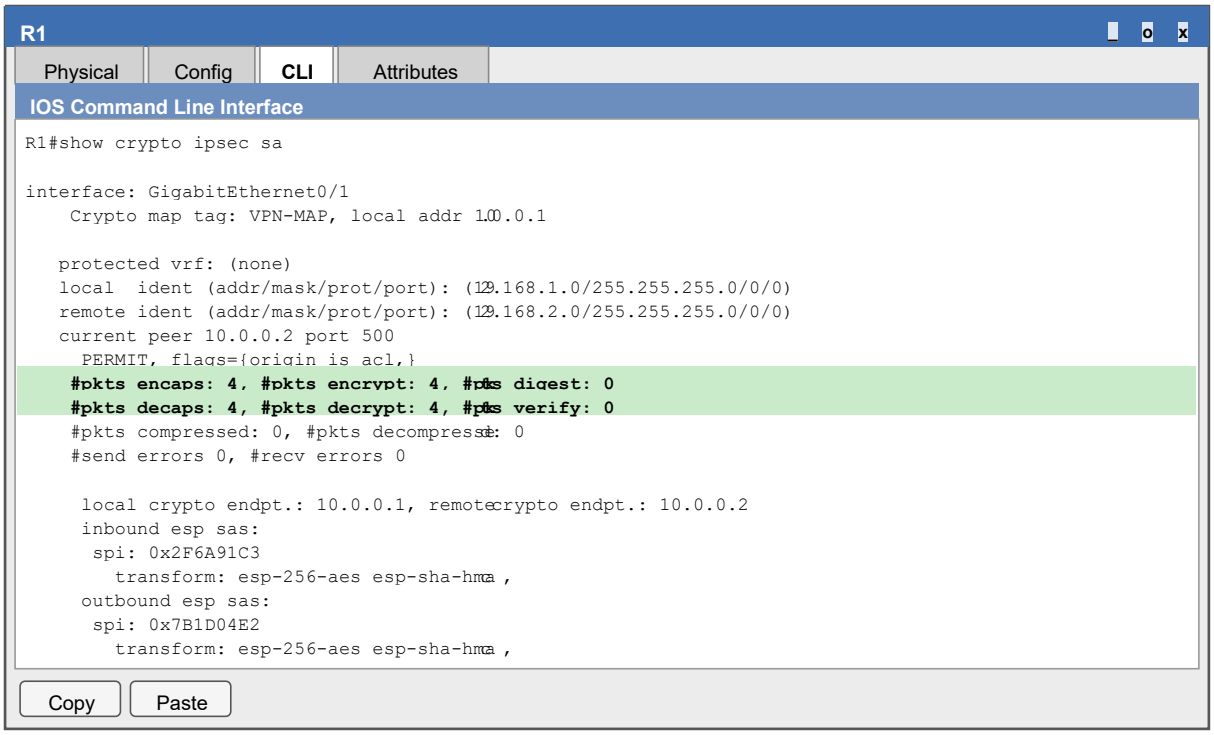
Input 1: generate interesting traffic. On PC1 run `ping 192.168.2.10` twice. The first request(s) may time out while IKE negotiates.



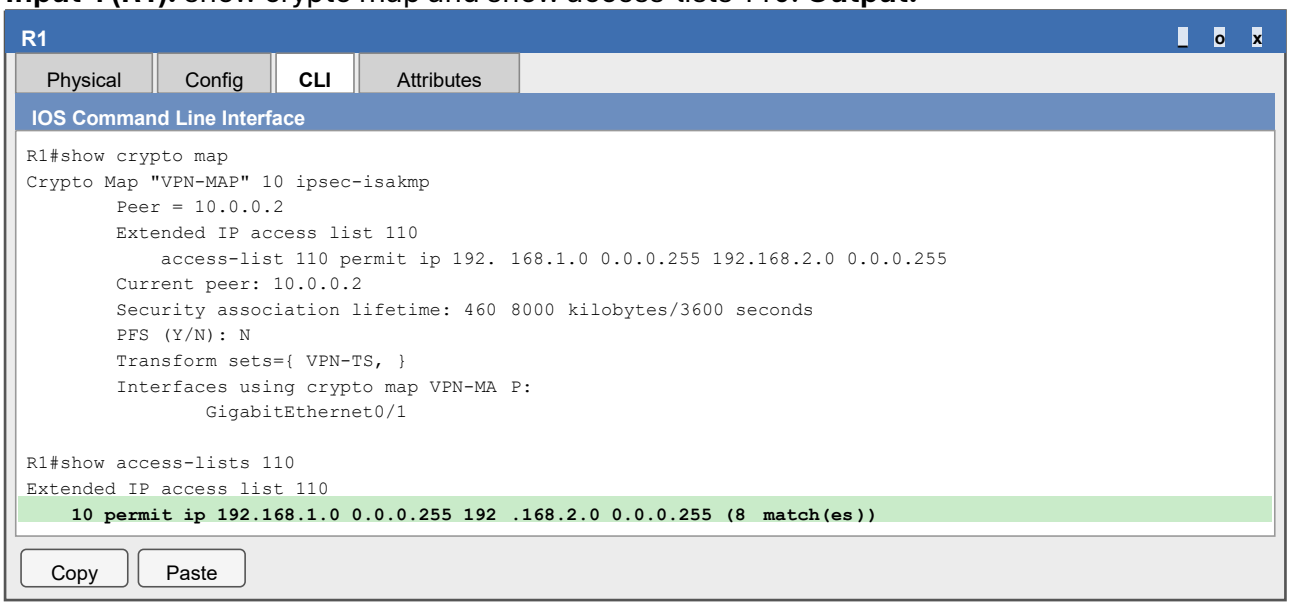
Input 2 (R1): show crypto isakmp sa. **Output:** Phase 1 is up when the state is **QM_IDLE**.



Input 3 (R1): show crypto ipsec sa. Output: the encaps/encrypt and decaps/decrypt counters must be greater than 0.



Input 4 (R1): show crypto map and show access-lists 110. **Output:**



Step 9: Compare the packet before and after IPsec

Input (optional, Simulation mode): click Simulation, Edit Filters > tick ICMP and ISAKMP (and ESP/IPsec if listed), use Add Simple PDU from PC1 to PC2 and press Capture/Forward until the packet is on the R1-R2 link. To see IKE again, run clear crypto isakmp and clear crypto sa on both routers first. Repeating CDP packets between the routers are normal neighbour advertisements and are not IPsec.

Output: the concept below. Event names in the packet viewer differ between Packet Tracer versions, so the counters in show crypto ipsec sa remain your main proof.

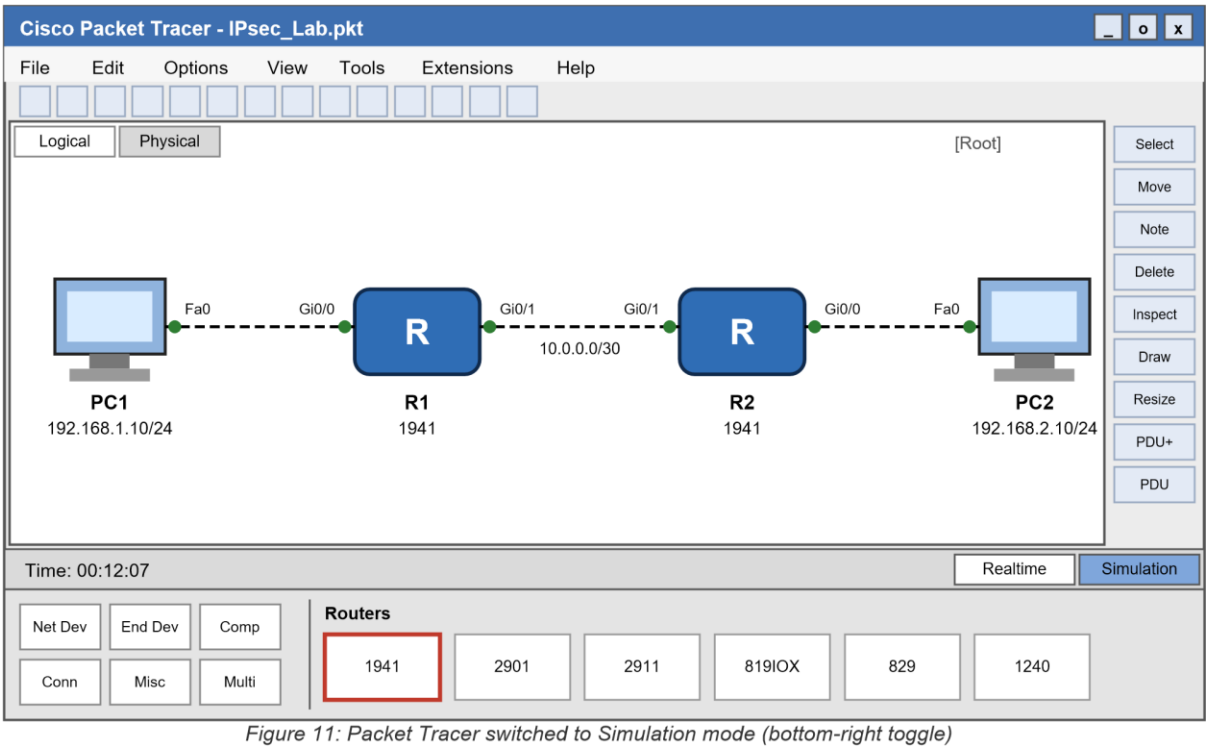
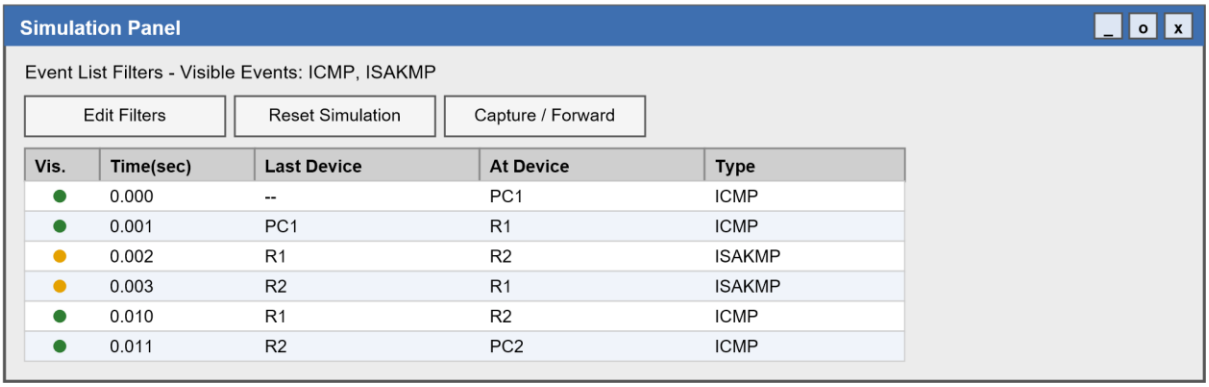


Figure 11: Packet Tracer switched to Simulation mode (bottom-right toggle)



Vis.	Time(sec)	Last Device	At Device	Type
●	0.000	--	PC1	ICMP
●	0.001	PC1	R1	ICMP
●	0.002	R1	R2	ISAKMP
●	0.003	R2	R1	ISAKMP
●	0.010	R1	R2	ICMP
●	0.011	R2	PC2	ICMP

Before IPsec (plain packet on the R1-R2 link)

IP header 192.168.1.10 -> 192.168.2.10	**ICMP** Echo request	**Data** readable by anyone

After IPsec (ESP tunnel mode on the R1-R2 link)

New IP header 10.0.0.1 -> 10.0.0.2	**ESP header** SPI + sequence no.	**ENCRYPTED (AES-256)** original IP header + ICMP + data	**ESP trailer**	**ESP auth** SHA-HMAC

Outside the routers (on the LANs) the packet is unchanged. Only on the 10.0.0.0/30 link between R1 and R2 is it wrapped in ESP, so an eavesdropper sees only the two router addresses, the SPI and unreadable data.

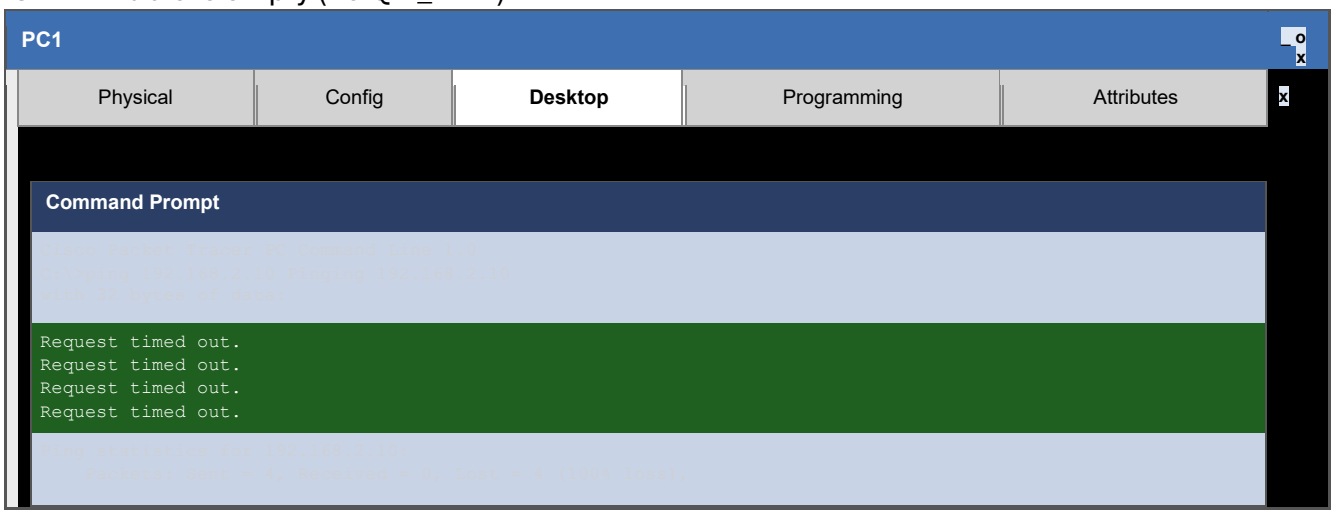
Step 10: Negative test, protection against an unauthorized peer

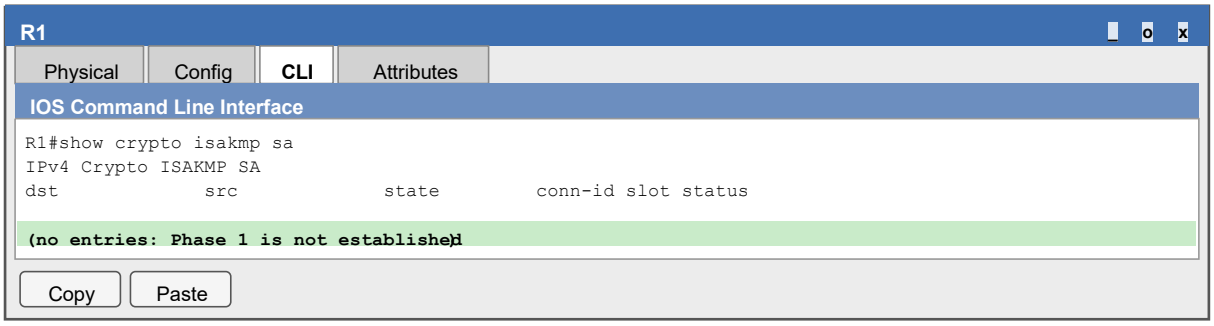
Input: make R2 use a wrong key, then clear the tunnel on both routers.

```
R2# configure terminal
R2(config)# no crypto isakmp key Lab@IPsec2026 address 10.0.0.1
R2(config)# crypto isakmp key WrongKey123 address 10.0.0.1
R2(config)# end

R1# clear crypto isakmp
R1# clear crypto sa
R2# clear crypto isakmp
R2# clear crypto sa
```

Now ping from PC1 and check Phase 1 on R1. Output: all requests time out and the ISAKMP table is empty (no QM_IDLE).

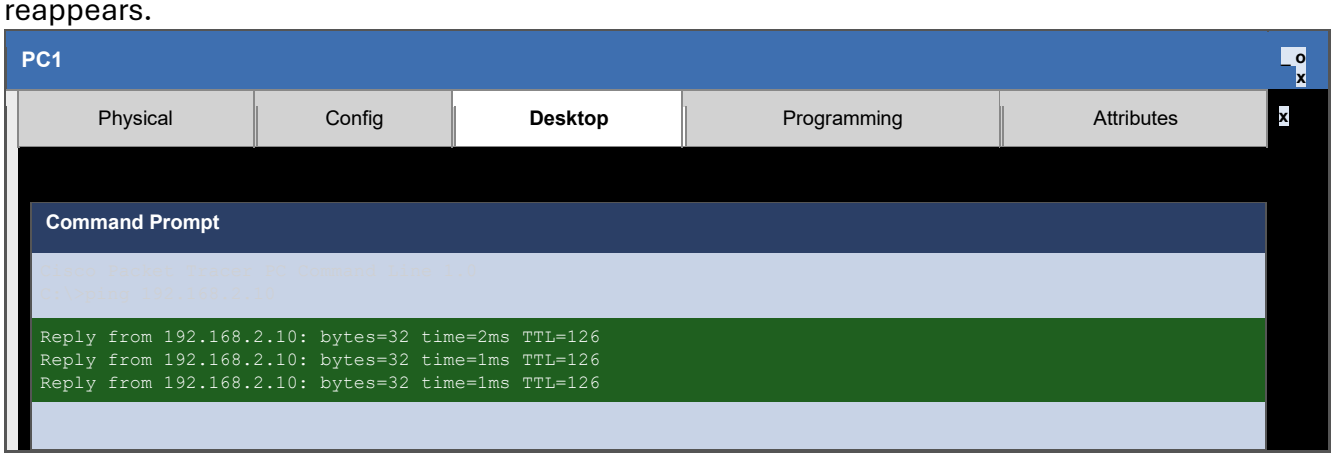


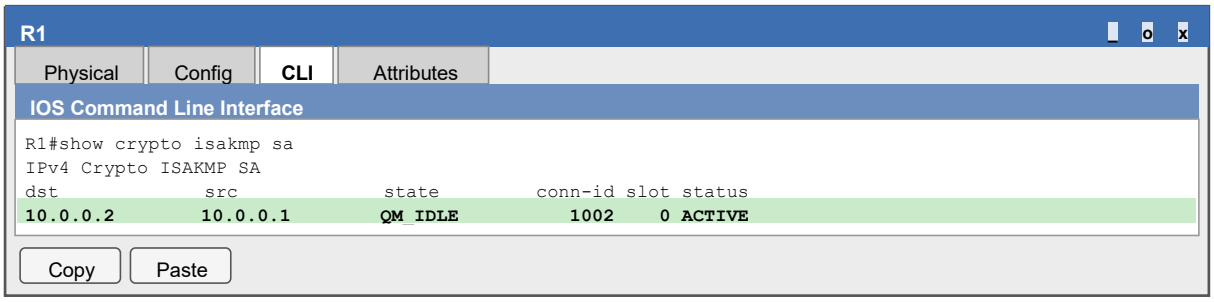


Input (restore the correct key on R2):

```
R2# configure terminal
R2(config)# no crypto isakmp key WrongKey123 address 10.0.0.1
R2(config)# crypto isakmp key Lab@IPsec2026 address 10.0.0.1
R2(config)# end
R2# clear crypto isakmp
R2# clear crypto sa
```

Ping again from PC1 (the first request may time out). **Output:** replies return and QM_IDLE reappears.





Step 11: Save the work

1. File > Save As > IPsec_Lab.pkt.
2. On each router run show running-config and copy the crypto parts into your journal.
3. Solution: Final Configuration of Both Routers

Compare your configuration with the relevant parts of the final running-config below. (Defaults such as hash sha and lifetime 86400 may be hidden by the router because they are default values.)

R1

Physical | Config | **CLI** | Attributes

IOS Command Line Interface

```
hostname R1
!
crypto isakmp policy 10
 encr aes 256
 authentication pre-share
 group 5
crypto isakmp key Lab@IPsec2026 address 100.0.2
!
crypto ipsec transform-set VPN-TS esp-aes256 esp-sha-hmac
!
crypto map VPN-MAP 10 ipsec-isakmp
 set peer 10.0.0.2
 set transform-set VPN-TS
 match address 110
!
interface GigabitEthernet0/0
 ip address 192.168.1.1 255.255.255.0
interface GigabitEthernet0/1
 ip address 10.0.0.1 255.255.255.252
 crypto map VPN-MAP
!
ip route 192.168.2.0 255.255.255.0 10.0.02
access-list 110 permit ip 192.168.1.0 0.001.255 192.168.2.0 0.0.0.255
```

Copy | Paste

Figure 18: Solution: Router R1

R2

Physical | Config | **CLI** | Attributes

IOS Command Line Interface

```
hostname R2
!
crypto isakmp policy 10
 encr aes 256
 authentication pre-share
 group 5
crypto isakmp key Lab@IPsec2026 address 100.0.1
!
crypto ipsec transform-set VPN-TS esp-aes256 esp-sha-hmac
!
crypto map VPN-MAP 10 ipsec-isakmp
 set peer 10.0.0.1
 set transform-set VPN-TS
 match address 110
!
interface GigabitEthernet0/0
 ip address 192.168.2.1 255.255.255.0
interface GigabitEthernet0/1
 ip address 10.0.0.2 255.255.255.252
 crypto map VPN-MAP
!
ip route 192.168.1.0 255.255.255.0 10.0.01
access-list 110 permit ip 192.168.2.0 0.001.255 192.168.1.0 0.0.0.255
```

Copy | Paste

Figure 19: Solution: Router R2

Practical No. 7

Aim: Configure and implement secure web communication using SSL/TLS protocols, including certificate management and secure session establishment.

Theory:

Secure Socket Layer

Secure Sockets Layer (SSL) is an Internet security protocol that encrypts data to ensure secure communication between devices over a network.

- Developed by Netscape in 1995, SSL provides privacy, authentication and data integrity for online communications.
- It is the predecessor of TLS (Transport Layer Security), which is now the standard protocol for secure communications on the Internet.

SSL Protocols

SSL consists of several protocols, each handling a different aspect of secure communication

1. SSL Record Protocol

- Provides confidentiality and message integrity.
- Application data is divided into fragments, optionally compressed and appended with a Message Authentication Code (MAC).
- The data is then encrypted and transmitted with an SSL header.

2. Handshake Protocol

Establishes SSL sessions and authenticates clients and servers.

Four phases:

- Client and server exchange hello packets, protocol versions and cipher suites.
- Server sends its certificate and server key information.
- Client responds with its certificate and key exchange.
- Change Cipher Spec finalizes the handshake, activating secure communication.

3. Change-Cipher Spec Protocol.

- Signals that pending cryptographic parameters from the handshake should now become active.
- Consists of a single 1-byte message.

4. Alert Protocol

- Communicates SSL-related warnings or errors.
- **Warning alerts (level 1):** Non-critical issues, such as expired or unsupported certificates.
- **Fatal alerts (level 2):** Critical errors, such as handshake failures, bad record MAC or illegal parameters, which terminate the connection.

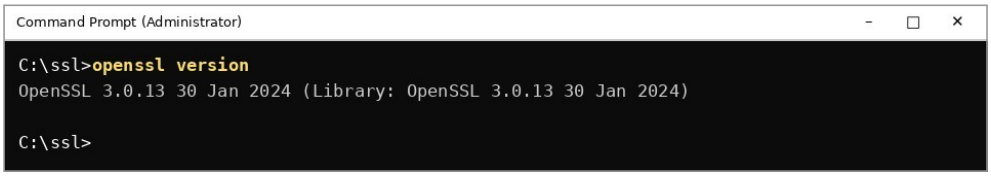
Procedure:

Step 1: Check that OpenSSL is installed

INPUT

```
openssl version
```

OUTPUT



Step 2: Generate the server's private key (2048-bit RSA)

INPUT

```
openssl genrsa -out server.key 2048
openssl rsa -in server.key -noout -
check
```

OUTPUT

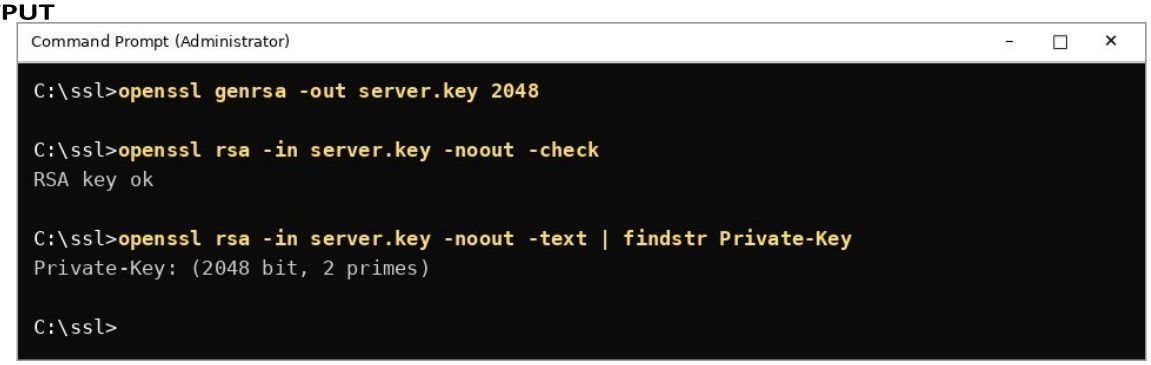


Fig. 3: Private key created and verified ('RSA key ok')

Step 3: Create a Certificate Signing Request (CSR)

INPUT

```
openssl req -new -key server.key -out server.csr
```

Enter these details when asked (press Enter to skip optional ones):

Prompt	Input to type	Meaning
Country Name	IN	2-letter country code
State or Province	Maharashtra	Your state
Locality Name	Mumbai	Your city
Organization Name	Test College	Your organisation (any name for the lab)
Organizational Unit	BSc IT	Department
Common Name	**localhost**	**Must match the address used in the browser**
Email / challenge password / company name	(press Enter)	Optional

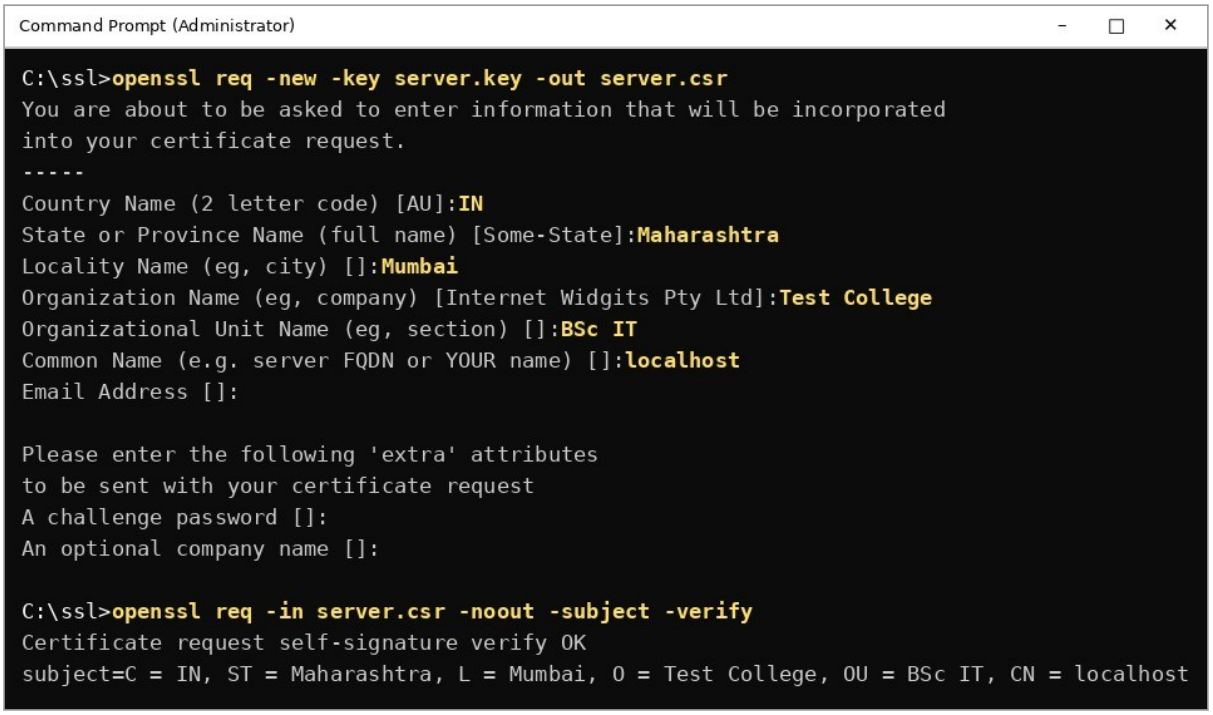


Fig. 4: CSR created and its signature verified

Step 4: Create the self-signed certificate and inspect it

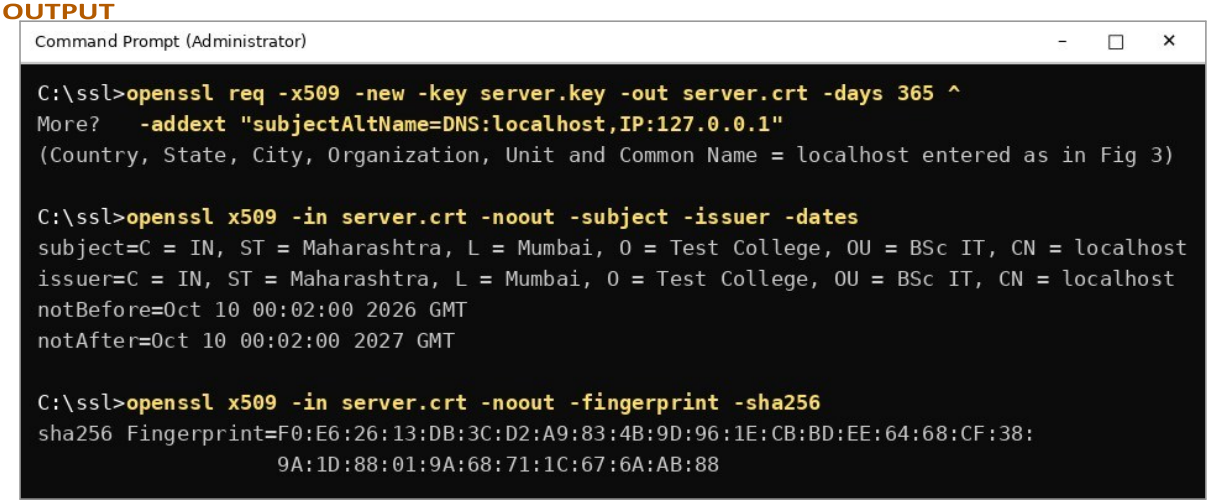
A self-signed certificate is signed with our own key instead of a CA's key. The -addext part adds a Subject Alternative Name (SAN), which modern browsers require; without it Chrome/Edge reject the certificate even for localhost.

INPUT

```
openssl req -x509 -new -key server.key -out server.crt -
days 365 ^   -addext
"subjectAltName=DNS:localhost,IP:127.0.0.1" openssl x509 -
in server.crt -noout -subject -issuer -dates openssl x509
-in server.crt -noout -fingerprint -sha256
```

The ^ at the end of a line tells Command Prompt the command continues on the next line (you may also type it all on one line). Enter the same details as Step 3.

OUTPUT



Step 5: Write and start the HTTPS server

Create index.html in C:\ssl containing <h1>Secure Page</h1>, and save the following as https_server.py:

INPUT

```
import http.server, ssl

server_address = ('localhost', 4443) httpd =
http.server.HTTPServer(server_address, http.server.SimpleHTTPRequestHandler)

context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER) context.minimum_version =
ssl.TLSVersion.TLSv1_2   # block old, weak TLS versions
context.load_cert_chain(certfile="server.crt", keyfile="server.key")
httpd.socket = context.wrap_socket(httpd.socket, server_side=True)

print("Serving HTTPS on https://localhost:4443")
httpd.serve_forever()
```

Run it (keep this window open):

```
python https_server.py
```

OUTPUT



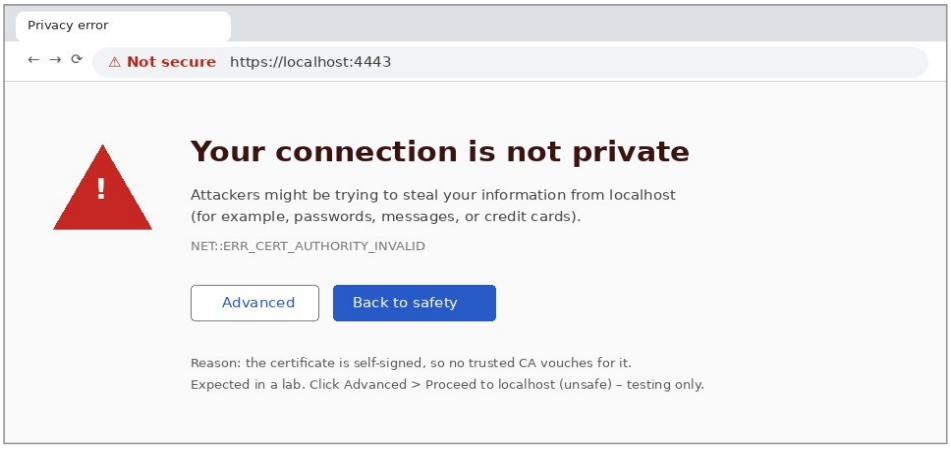
Step 6: Test in the browser

INPUT

Open https://localhost:4443/index.html in Chrome or Edge.

OUTPUT

The browser first shows a warning because the certificate is self-signed and not issued by a trusted CA. This is the expected result and shows certificate verification at work:



Click Advanced > Proceed to localhost (unsafe). This is acceptable only for your own lab certificate. The page loads over HTTPS; click the padlock / 'Not secure' label > Certificate to view the details:

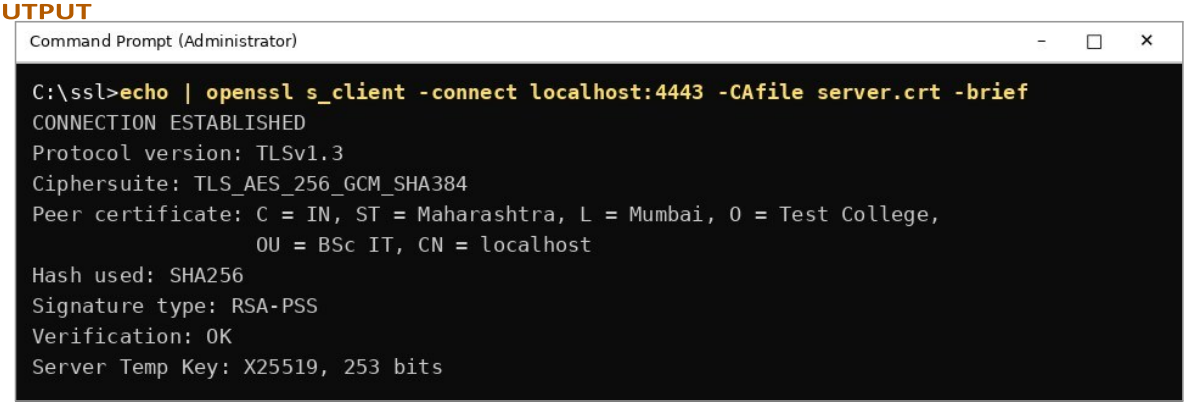


Step 7: Verify the secure session (handshake) from the command line

INPUT

```
echo | openssl s_client -connect localhost:4443 -CAfile server.crt -brief
```

OUTPUT

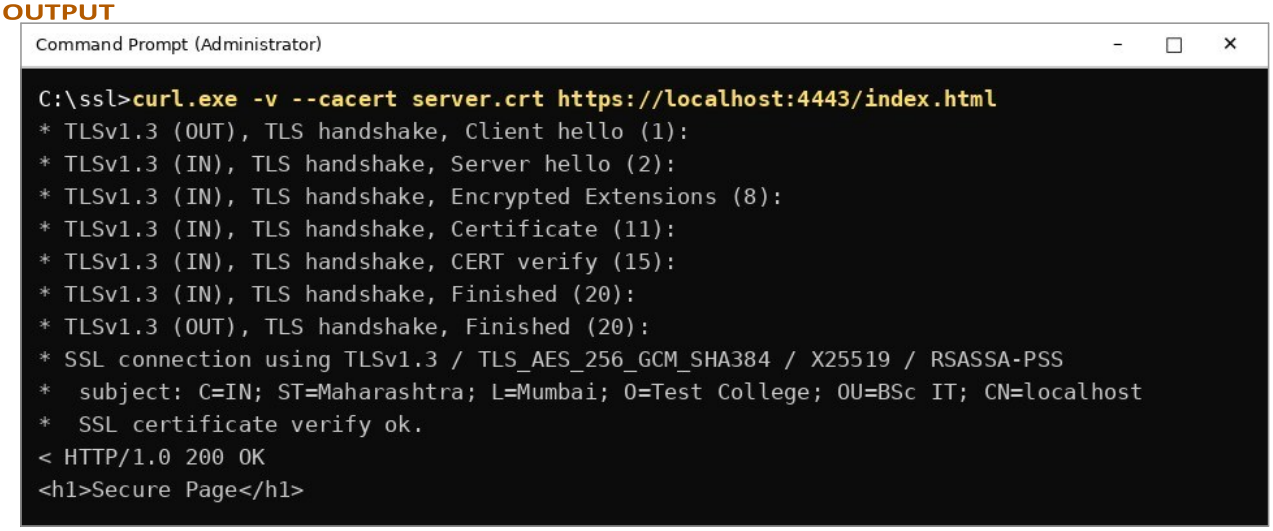


messages step by step with curl (matches Fig. 1):

INPUT

```
curl.exe -v --cacert server.crt https://localhost:4443/index.html
```

OUTPUT



Step 8: Secure client in Python (certificate verification enabled)

Save as client.py. The client trusts only our certificate, so it verifies the server before sending data

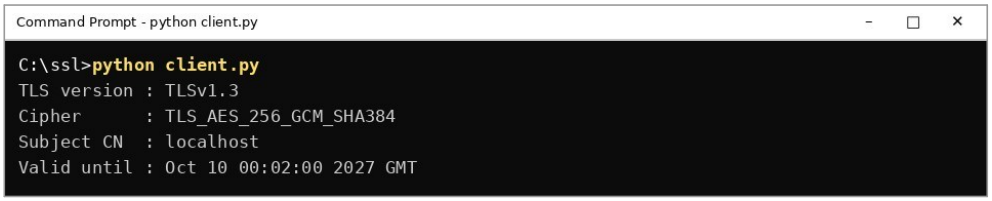
INPUT

```
import ssl, socket

ctx = ssl.create_default_context(cafile="server.crt") with
socket.create_connection(("localhost", 4443)) as s:     with
ctx.wrap_socket(s, server_hostname="localhost") as t:
        print("TLS version :", t.version())         print("Cipher      :",
t.cipher()[0])         c = t.getpeercert()         print("Subject CN  :",
dict(x[0] for x in c['subject'])['commonName'])         print("Valid until
:", c['notAfter'])
```

```
python client.py
```

OUTPUT



Step 9: Negative tests: show that security checks really work

Test A - a client that does not trust the certificate must refuse to connect. Test B - a client that only supports TLS 1.0/1.1 must be rejected by the server (it enforces TLS 1.2+). For Test B save old_client.py (results can vary on your PC because newer Python/OpenSSL builds may refuse to even attempt TLS 1.0/1.1; that is also a valid observation):

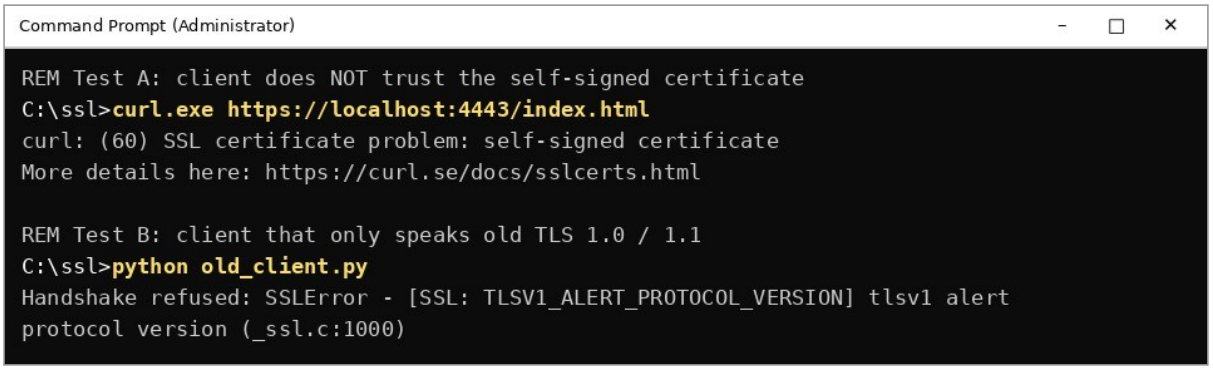
INPUT

```
import ssl, socket

ctx =
ssl.SSLContext(ssl.PROTOCOL_TLS_CLIENT)
ctx.check_hostname = False ctx.verify_mode =
ssl.CERT_NONE ctx.minimum_version =
ssl.TLSVersion.TLSv1 ctx.maximum_version =
ssl.TLSVersion.TLSv1_1 try:
    ctx.set_ciphers("DEFAULT:@SECLEVEL=0")     with
socket.create_connection(("localhost", 4443)) as s:         with
ctx.wrap_socket(s, server_hostname="localhost") as t:
            print("connected", t.version())
except Exception as e:
    print("Handshake refused:", type(e).__name__, "-", str(e)[:90])
```

```
curl.exe https://localhost:4443/index.html
python old_client.py
```

OUTPUT:


Command Prompt (Administrator)
REM Test A: client does NOT trust the self-signed certificate
C:\ssl>curl.exe https://localhost:4443/index.html
curl: (60) SSL certificate problem: self-signed certificate
More details here: https://curl.se/docs/sslcerts.html
REM Test B: client that only speaks old TLS 1.0 / 1.1
C:\ssl>python old_client.py
Handshake refused: SSLError - [SSL: TLSV1_ALERT_PROTOCOL_VERSION] tlsv1 alert
protocol version (_ssl.c:1000)

Practical No. 8

Aim: Set up and configure an intrusion detection system (IDS) to monitor network traffic and detect potential security breaches or malicious activities.

Theory:

Intruders

An **intruder** in cybersecurity is any unauthorized person or software program that breaches a system, network, or data repository to compromise confidentiality, integrity, or availability.

Types of Intruders

- **Masquerader:** An unauthorized outsider who penetrates system access controls to pose as a legitimate user.
- **Misfeasor:** A legitimate user who exceeds their authorized permissions or misuses their privileges.
- **Clandestine User:** An administrator or individual with high-level supervisory control who seizes system control to bypass audits and security mechanisms.

Core Intrusion Detection & Analysis Techniques

- **Signature-Based Detection:** Compares network packets or system logs against a database of known attack signatures and fingerprints.
- **Anomaly-Based Detection:** Builds a baseline of normal user or network behavior using machine learning or statistical analysis and flags major deviations as potential unknown threats.
- **Stateful Protocol Analysis:** Tracks active communication states and command sequences (such as FTP or HTTP) to catch behavior that breaks standard protocol rules
- **Heuristic-Based Detection:** Uses algorithmic rules and "educated guesses" to catch modified or polymorphic malware that lacks a direct signature match.
- **Reputation-Based Detection:** Evaluates traffic and communication based on the external reputation score of the remote IP address or host.

Procedure:

Step 1: Install Npcap

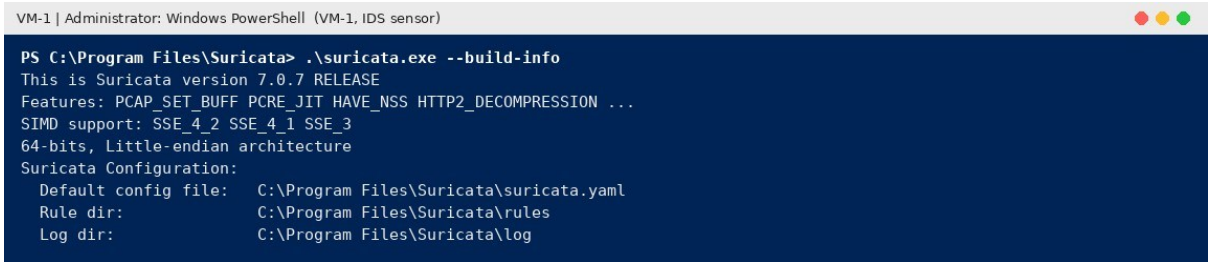
- Run the Npcap installer as Administrator and accept defaults (it is the packet-capture driver Suricataneeds).
- Reboot if the installer asks.

Step 2: Install Suricata

- Run the Suricata MSI installer and accept the default path: C:\Program Files\Suricata.

Open **PowerShell as Administrator** and check the install:

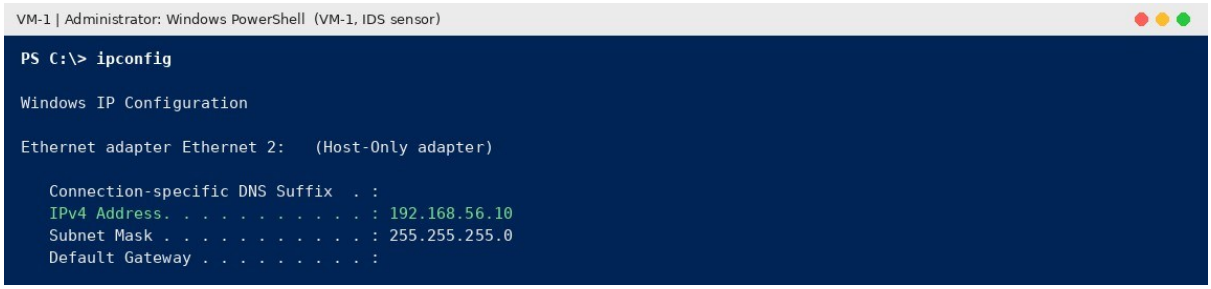
```
cd "C:\Program Files\Suricata"
.\suricata.exe --build-info
```



Step 3: Find your network interface

```
ipconfig # Note the IPv4 address of the Host-Only adapter, e.g.
192.168.56.10
.\suricata.exe --list-interfaces
```

On Windows you can give Suricata either the interface's IP address or its \Device\NPF_{...} name with the -i option.



Step 4: Configure suricata.yaml (minimal changes)

Make a backup copy first, then open the file in Notepad (run as Administrator) and change only these items:

```
copy suricata.yaml suricata.yaml.bak
notepad suricata.yaml
```

Step 5: Create your own rules file

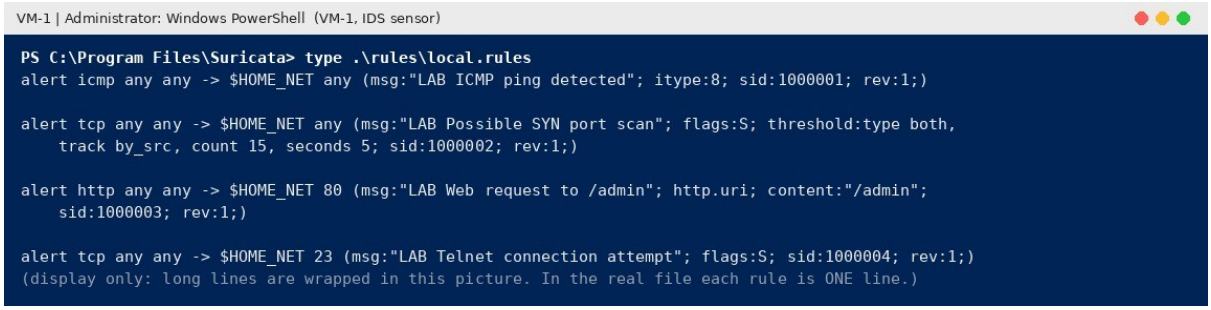
Create local.rules in the rules folder (C:\Program Files\Suricata\rules\local.rules). Each rule must be on **one single line**. Use a unique **sid** above 1000000.

```
alert icmp any any -> $HOME_NET any (msg:"LAB ICMP ping detected"; itype:8;
sid:1000001; rev:1;)

alert tcp any any -> $HOME_NET any (msg:"LAB Possible SYN port scan"; flags:S;
threshold:type both, track by_src, count 15, seconds 5; sid:1000002; rev:1;)

alert http any any -> $HOME_NET 80 (msg:"LAB Web request to /admin"; http.uri;
content:"/admin"; sid:1000003; rev:1;)

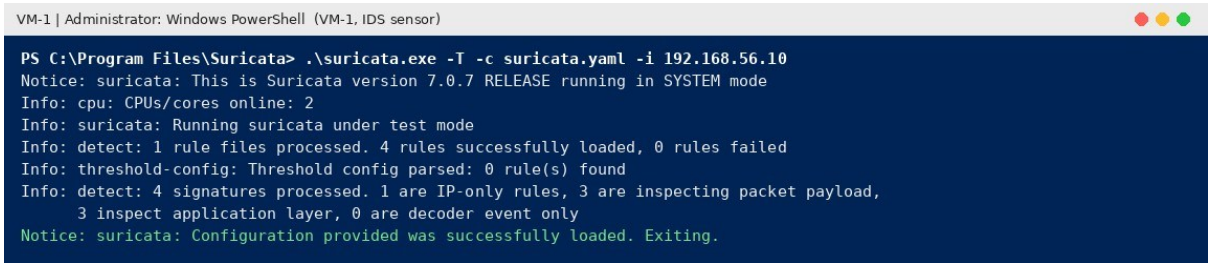
alert tcp any any -> $HOME_NET 23 (msg:"LAB Telnet connection attempt"; flags:S;
sid:1000004; rev:1;)
```



Step 6: Test the configuration

```
.\suricata.exe -T -c suricata.yaml -i 192.168.56.10
```

The output should end with a message that the configuration was successfully loaded. If you see errors, read the line number mentioned, fix the typo (missing semicolon or bracket) and run the test again.

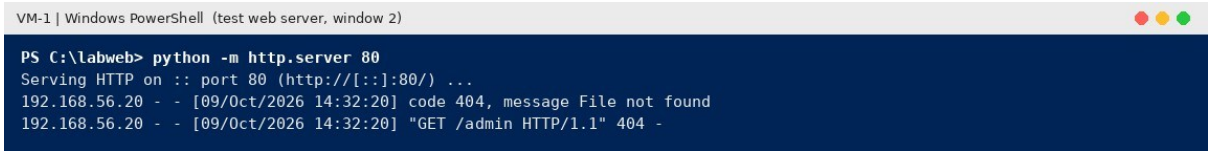


Step 7: Start a test web server on VM-1 (for rule 3)

Open a second PowerShell window on VM-1 and run:

```
mkdir C:\labweb cd
C:\labweb python -m
http.server 80
```

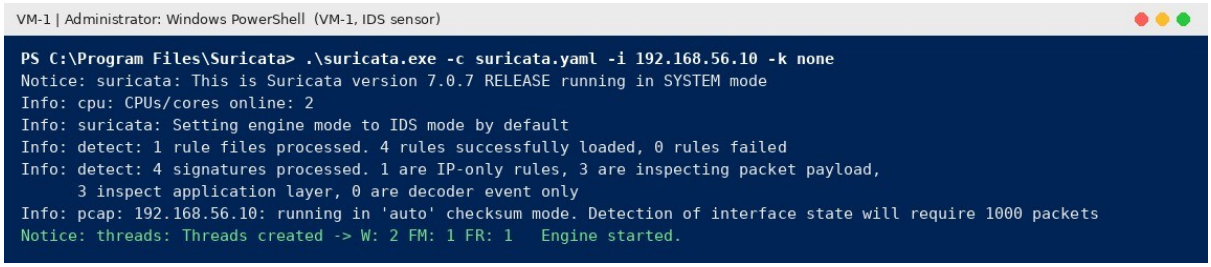
Windows Firewall may ask for permission: allow it on the **Private** network only.



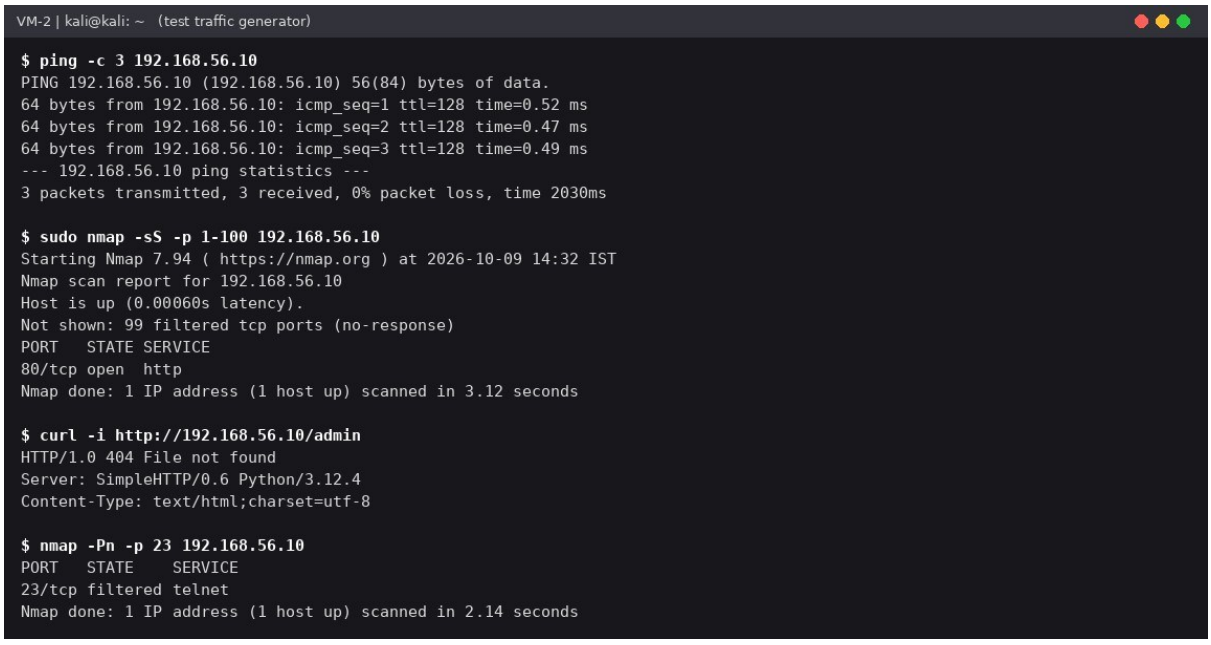
Step 8: Start Suricata

```
cd "C:\Program Files\Suricata" .\suricata.exe -c
suricata.yaml -i 192.168.56.10 -k none
```

-k none turns off checksum validation, which avoids missed alerts in virtual machines. Leave this window running; you should see the message that the engine has started and is processing packets.



Step 9: Generate safe test traffic from VM-2



Step 10: View the alerts

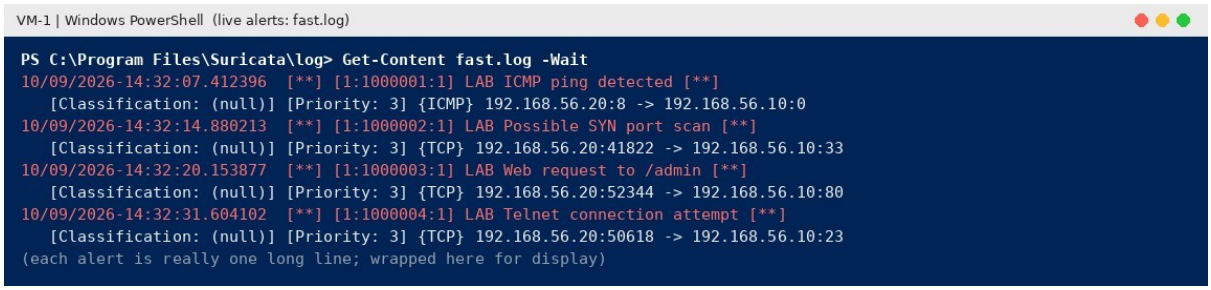
Open a third PowerShell window on VM-1 and watch the alert log live:

```
cd "C:\Program Files\Suricata\log" Get-
Content fast.log -Wait
```

A typical alert line looks like this (values will differ):

```
10/04/2026-14:32:07.123456 [**] [1:1000001:1] LAB ICMP ping detected [**]
[Classification: (null)] [Priority: 3] {ICMP} 192.168.56.20:8 -> 192.168.56.10:0
```

Read it as: timestamp, rule **[gid:sid:rev]**, message, priority, protocol, **source -> destination**. The detailed JSON log is eve.json in the same folder; open it with Notepad to see packet details.





Step 11: Stop and clean up

- Press Ctrl+C in the Suricata window and the web server window.
- Take screenshots of alerts for your journal, then delete C:\labweb if no longer needed.
- Remove the Windows Firewall exception created in Step 7 if you do not need it.

Practical No. 9

Aim: Analyze and identify malware samples using antivirus tools, analyze their behavior, and develop countermeasures to mitigate their impact.

Theory:

Viruses and Related Threats

A computer virus is a malicious program that attaches itself to a legitimate file or program, spreads by copying itself when that host is run, and can damage data or system operation. “Related threats” is the wider family of malicious software (malware) and attacks that behave similarly.

How infections spread

- Email attachments and links
- Infected downloads and pirated software
- Removable drives (USB)
- Network vulnerabilities and unpatched software
- Malicious websites (drive-by downloads)

Antivirus detection methods

- **Signature-based:** Matches known virus patterns. Fast, but misses new or polymorphic viruses.
- **Heuristic:** Looks for suspicious code structure or behavior.
- **Behavior monitoring:** Watches what a program does at runtime.
- **Sandboxing:** Runs suspicious files in an isolated environment to observe them.

Countermeasures

- Install and update antivirus software.
- Patch the OS and applications regularly.
- Do not open unknown attachments or links.
- Use a firewall and least-privilege accounts.
- Keep regular offline backups (the best defense against ransomware).
- Use multi-factor authentication and user awareness training.

Procedure:

Step 1: Lab Setup

1. INPUT (what you do / type)
2. Install the Windows VM and all tools above.
3. VM settings: **Network** = Host-only (or Disconnected), **Shared Folders** = off, **Clipboard / Drag-and-Drop** = Disabled.

Create the lab folders in PowerShell:

```
New-Item -ItemType Directory -Force -Path C:\MalwareLab\Samples, C:\MalwareLab\Reports | Out-Null
```

4. Take a VM snapshot named **Clean-Baseline**.

Step 2: Antivirus Detection Using the EICAR Test File

INPUT (what you do / type)

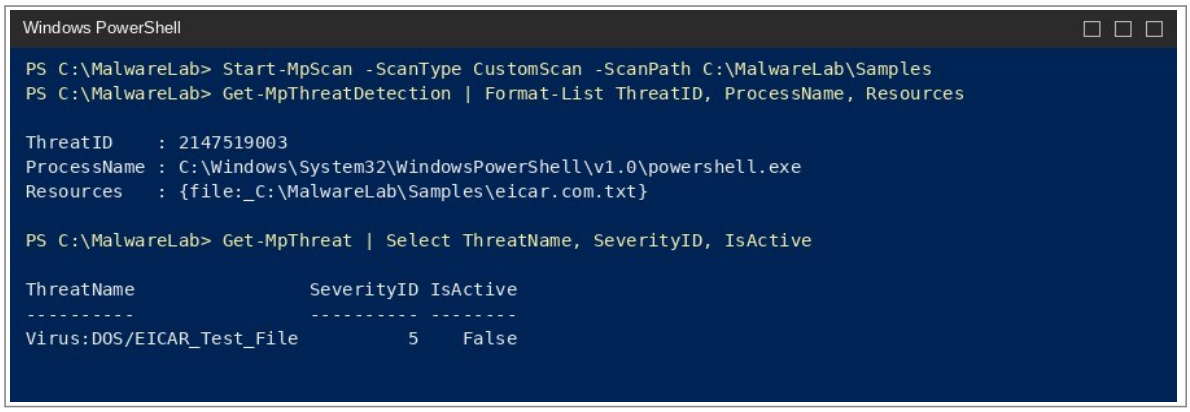
1. Open **Windows Security > Virus & threat protection** and make sure Real-time protection is **On**.
2. Save the EICAR test file into **C:\MalwareLab\Samples** from the official page.
3. Observe the alert and open **Protection history**.
4. Run a custom scan and list the detections:

```
Start-MpScan -ScanType CustomScan -ScanPath C:\MalwareLab\Samples
Get-MpThreatDetection | Format-List ThreatID, ProcessName, Resources
Get-MpThreat | Select ThreatName, SeverityID, IsActive
```

OUTPUT (what you should see)



Figure 1: Windows Security - Protection history showing the EICAR file quarantined (sample illustration).



Step 3: Hash Reputation Check with VirusTotal

A **hash** is a unique fingerprint of a file. Analysts search the hash instead of uploading the file, which is safer and protects privacy.

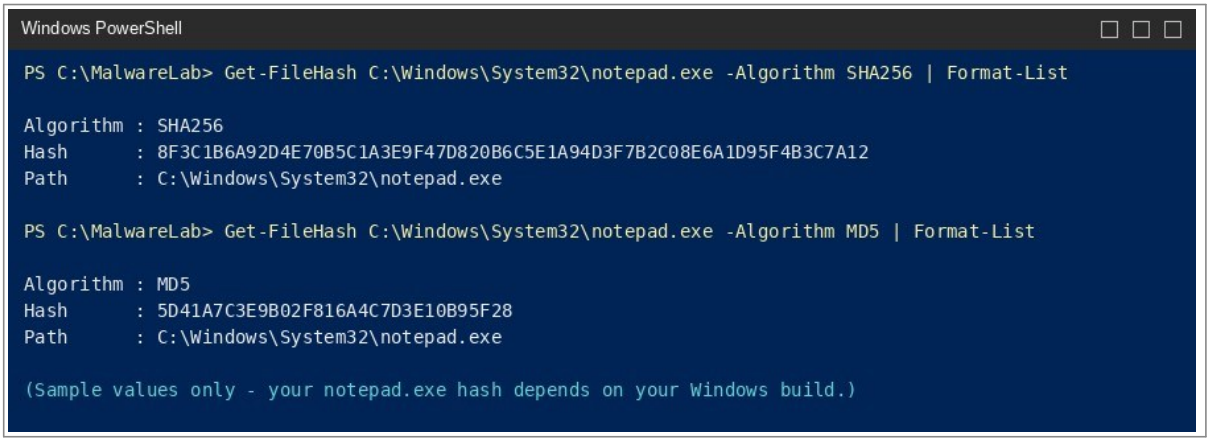
INPUT (what you do / type)

1. Compute hashes of a normal Windows program (known-clean reference):

```
Get-FileHash C:\Windows\System32\notepad.exe -Algorithm SHA256 | Format-List
Get-FileHash C:\Windows\System32\notepad.exe -Algorithm MD5    | Format-List
```

2. Paste the SHA-256 value into the search box on **virustotal.com**.
3. Search the official EICAR SHA-256 hash (published on eicar.org):
   `275a021bbfb6489e54d471899f7db9d1663fc695ec2fe2a2c4538aabf651fd0f`
4. Note the detection ratio, file type and Details/Behavior/Community tabs.

OUTPUT (what you should see)



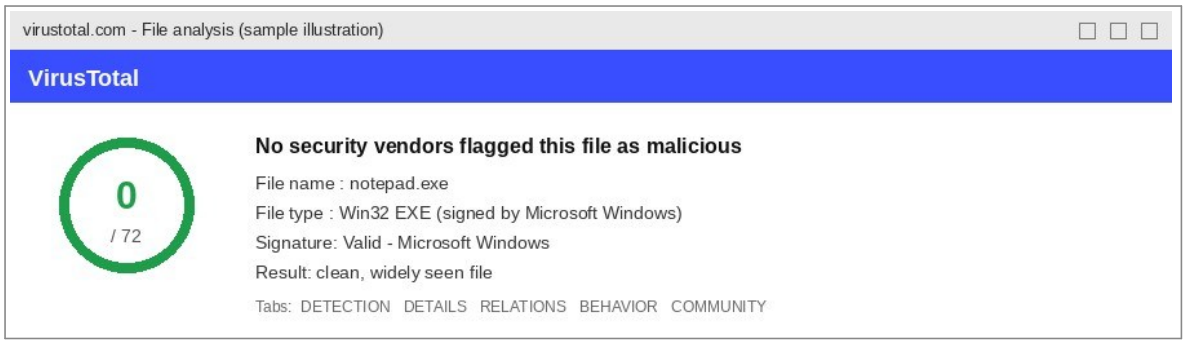
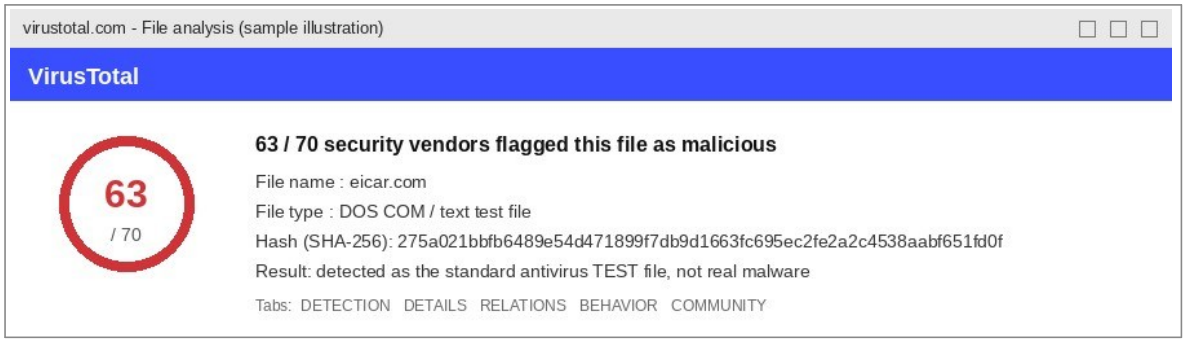


Figure 4: Clean file - 0 detections (sample illustration; counts change over time).



Step 4: Static Analysis (Without Running the File)

INPUT (what you do / type)

```
(Get-Item C:\Windows\System32\notepad.exe).VersionInfo |
  Format-List FileDescription,CompanyName,FileVersion
Get-AuthenticodeSignature C:\Windows\System32\notepad.exe |
  Format-List Status,StatusMessage Get-AuthenticodeSignature
C:\MalwareLab\Samples\SafeSim.ps1 | Format-List Status strings.exe -n 8
C:\Windows\System32\notepad.exe | more
```

Open **PEStudio**, drag notepad.exe into it, and review the **sections**, **imports** and **strings** panes. (Run the
SafeSim.ps1 signature command after you create the script in Part D.)

OUTPUT

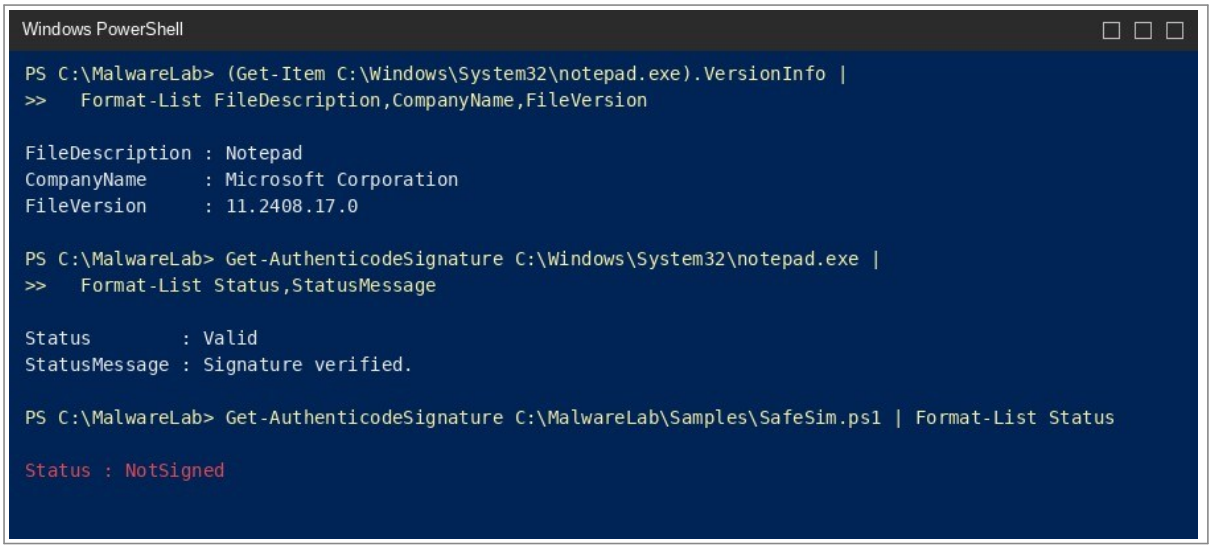


Figure 6: Version information and digital-signature checks (sample illustration).

PEStudio - sections view (illustrative sample)

Section	Virtual size	Raw size	Entropy	Meaning
.text	0x00041A2C	0x00041C00	6.41	Normal compiled code
.rdata	0x0001A3F0	0x0001A400	5.12	Read-only data, imports
.data	0x00003B10	0x00001200	3.08	Initialised data
.rsrc	0x000C2E40	0x000C3000	5.60	Icons, dialogs, strings
(packed sample)	.UPX0	...	7.92	Entropy near 8 = packed/encrypted

Imports flagged by PEStudio on suspicious files typically include: VirtualAllocEx, WriteProcessMemory, CreateRemoteThread, RegSetValueExW, URLDownloadToFileW. Clean notepad.exe shows none of the injection group.

Step 5: Dynamic Analysis of a Safe Behavior Simulator

INPUT (what you do / type)

Create C:\MalwareLab\Samples\SafeSim.ps1:

```
# SafeSim.ps1 - HARMLESS behaviour simulator for classroom use only
$dir = "$env:USERPROFILE\MalwareLab\SimDrop"
New-Item -ItemType Directory -Force -Path $dir | Out-Null

# Behaviour 1: drops a file
"This is a harmless lab file." | Out-File "$dir\dropped_note.txt"

# Behaviour 2: adds a startup entry (opens Notepad at logon)
$run = "HKCU:\Software\Microsoft\Windows\CurrentVersion\Run"
New-ItemProperty -Path $run -Name "LabSimUpdater" -Value "notepad.exe" `
    -PropertyType String -Force | Out-Null

# Behaviour 3: tries a web request (fails harmlessly if network is off) try { Invoke-
WebRequest "http://example.com" -UseBasicParsing -TimeoutSec 5 | Out-Null } catch { Write-
Host "Network request failed (expected if offline)" }

Start-Sleep -Seconds 2
Write-Host "SafeSim finished."
```

Start monitoring **before** running:

Process Monitor: Filter > *Process Name is powershell.exe* (Include). Clear and start capture.

Wireshark: choose the VM adapter, start capture, filter `dns || http`.

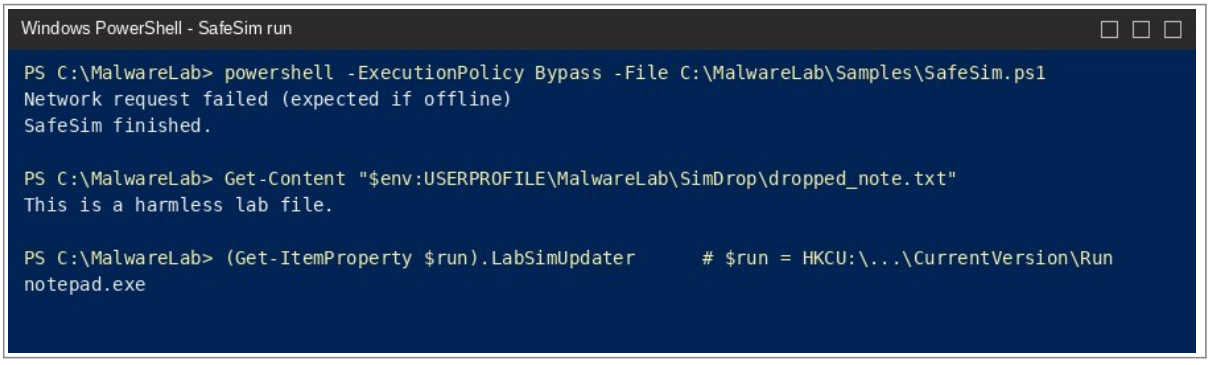
Autoruns: run it and use File > Save to store the **'before'** snapshot.

Run the simulator and verify its effects:

```
powershell -ExecutionPolicy Bypass -File C:\MalwareLab\Samples\SafeSim.ps1 Get-Content
"$env:USERPROFILE\MalwareLab\SimDrop\dropped_note.txt"
(Get-ItemProperty $run).LabSimUpdater
```

Stop the captures. In Autoruns use File > Compare and pick the saved 'before' file. Save the Procmon (.PML) and Wireshark (.pcapng) files to C:\MalwareLab\Reports.

OUTPUT (what you should see)



Process Monitor - Sysinternals (sample capture, filter: Process Name is powershell.exe)

Time	Process	PID	Operation	Path	Result
10:42:15	powershell.exe	4820	CreateFile	C:\Users\student\MalwareLab\SimDrop	SUCCESS
10:42:15	powershell.exe	4820	CreateFile	...\SimDrop\dropped_note.txt	SUCCESS
10:42:15	powershell.exe	4820	WriteFile	...\SimDrop\dropped_note.txt	SUCCESS
10:42:16	powershell.exe	4820	RegOpenKey	HKCU\...\CurrentVersion\Run	SUCCESS
10:42:16	powershell.exe	4820	RegSetValue	HKCU\...\Run\LabSimUpdater	SUCCESS
10:42:16	powershell.exe	4820	UDP Send	student-vm:51822 -> 192.168.56.1:domain	SUCCESS

Figure 9: Process Monitor showing file creation, registry write and a DNS send (sample illustration).

Autoruns - Compare against saved 'before' file (sample)

Autorun Entry	Description	Publisher	Image Path
HKCU\...\Run			
OneDrive	Microsoft OneDrive	Microsoft Corp.	c:\users\student\...\onedrive.exe
LabSimUpdater	Notepad	Microsoft Windows	c:\windows\system32\notepad.exe

Green row = entry added since the saved 'before' snapshot -> new persistence entry found.

Figure 10: Autoruns comparison highlighting the new LabSimUpdater entry (sample illustration).

Wireshark - Host-only adapter (display filter: dns || http)

dns || http

No.	Time	Source	Destination	Protocol	Info
1	0.000000	192.168.56.101	192.168.56.1	DNS	Standard query 0x1a2b A example.com
2	1.001300	192.168.56.101	192.168.56.1	DNS	Standard query 0x1a2b A example.com
3	3.002100	192.168.56.101	192.168.56.1	DNS	Standard query 0x1a2b A example.com

The VM tried to resolve example.com, but no reply arrives because the network is host-only.
Evidence of an attempted network connection = indicator of compromise (domain).

Step 6: Building Countermeasures

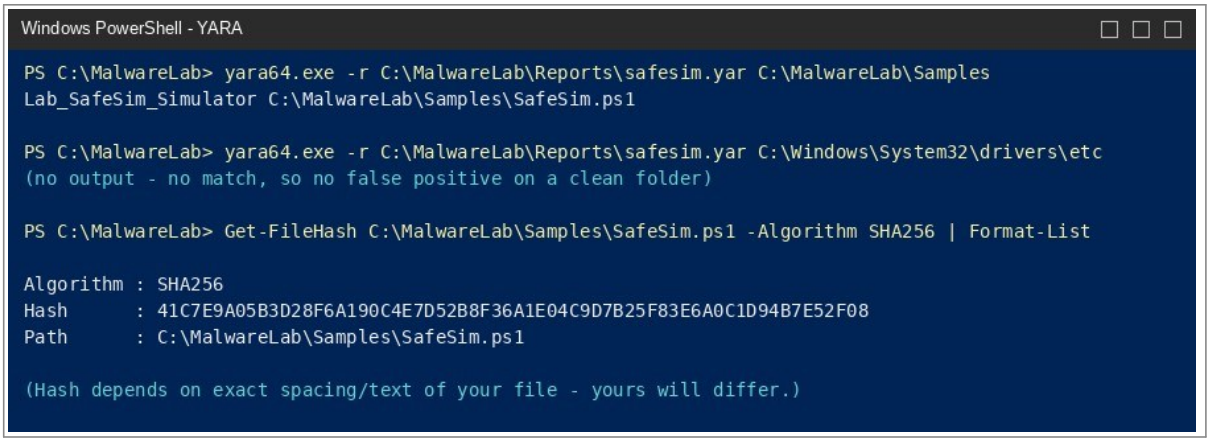
INPUT (what you do / type)

Save as **C:\MalwareLab\Reports\safesim.yar**:

```
rule
Lab_SafeSim_Simulator {
meta:
        description = "Detects the classroom SafeSim behaviour
simulator"         author      = "Student Name"     strings:         $a
= "LabSimUpdater" ascii wide
        $b = "MalwareLab"    ascii wide
$c = "SimDrop"       ascii wide
condition:
        all of them
}
```

```
yara64.exe   -r   C:\MalwareLab\Reports\safesim.yar   C:\MalwareLab\Samples   yara64.exe   -r
C:\MalwareLab\Reports\safesim.yar         C:\Windows\System32\drivers\etc         Get-FileHash
C:\MalwareLab\Samples\SafeSim.ps1 -Algorithm SHA256 | Format-List
```

OUTPUT (what you should see)



Step 7: Apply and verify mitigations

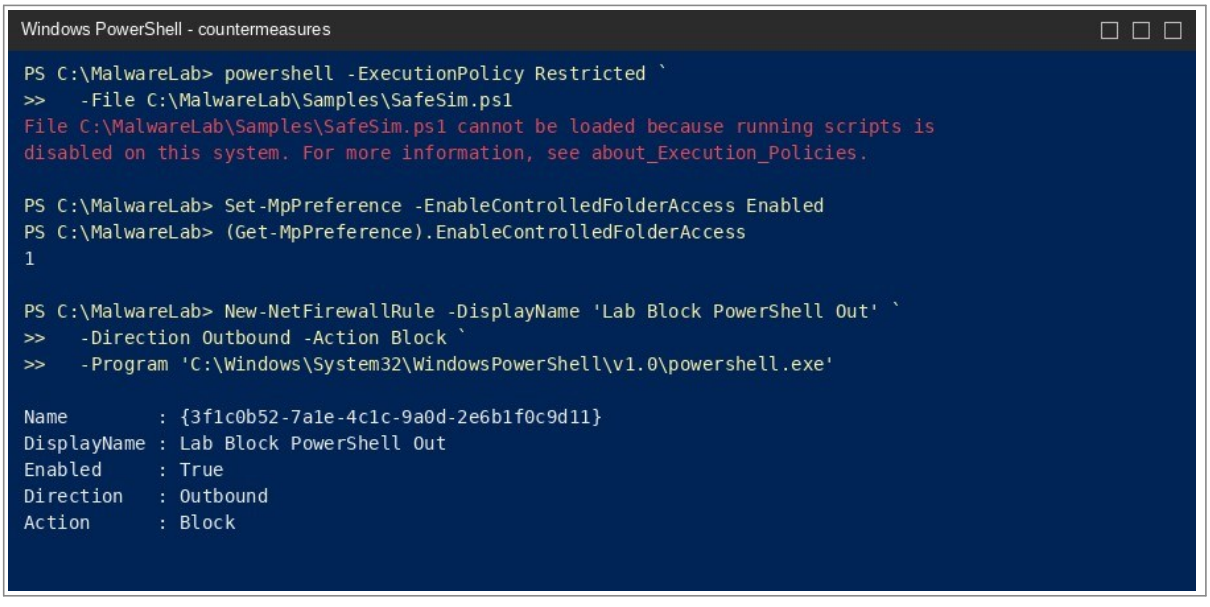
INPUT (what you do / type)

```
# 1. Restrict script execution, then try to run the simulator again
powershell -ExecutionPolicy Restricted -File
C:\MalwareLab\Samples\SafeSim.ps1

# 2. Ransomware protection (Controlled folder access)
Set-MpPreference -EnableControlledFolderAccess Enabled
(Get-MpPreference).EnableControlledFolderAccess

# 3. Outbound firewall rule that blocks PowerShell
New-NetFirewallRule -DisplayName 'Lab Block PowerShell Out' -Direction Outbound `
    -Action Block -Program 'C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe'
```

OUTPUT (what you should see)



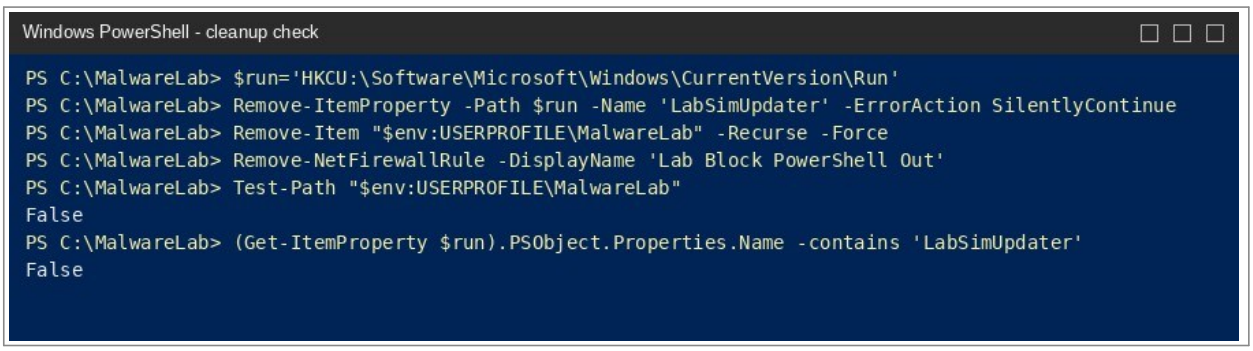
Step 8: Clean-up and Restoration

INPUT (what you do / type)

```
$run = "HKCU:\Software\Microsoft\Windows\CurrentVersion\Run"
Remove-ItemProperty -Path $run -Name "LabSimUpdater" -ErrorAction SilentlyContinue
Remove-Item "$env:USERPROFILE\MalwareLab" -Recurse -Force
Remove-NetFirewallRule -DisplayName 'Lab Block PowerShell Out'
Test-Path "$env:USERPROFILE\MalwareLab"
(Get-ItemProperty $run).PSObject.Properties.Name -contains 'LabSimUpdater'
```

If Controlled folder access was off before the lab, restore it with Set-MpPreference -EnableControlledFolderAccess Disabled. Copy out only your reports and screenshots (never the samples), then **revert the VM to 'Clean-Baseline'**. Note: the lab folders are under C:\MalwareLab and %USERPROFILE%\MalwareLab; delete C:\MalwareLab as well, or simply revert the snapshot.

OUTPUT (what you should see)



Practical No. 10

Aim: Configure and test firewall rules to control network traffic, filter packets based on specified criteria, and protect network resources from unauthorized access.

Theory:

Firewall
A firewall is a network security system that monitors and controls incoming and outgoing network traffic based on a set of security rules, acting as a barrier between a trusted internal network and an untrusted external network such as the Internet

Types of Firewalls

- **Packet filtering firewall:** Checks each packet's header (source/destination IP, port, protocol) against rules. It is fast and simple, but it does not track connections or look at content.
- **Stateful inspection firewall:** Tracks the state of active connections and allows only packets that belong to a valid, established connection. It is more secure than simple packet filtering.
- **Circuit-level gateway:** Works at the session layer and checks that a TCP handshake is valid before allowing the connection. It does not inspect the data itself.
- **Application-level gateway (proxy firewall):** Works at the application layer. The client connects to the proxy, and the proxy connects to the server on its behalf. It can inspect the actual content (HTTP, FTP, etc.), so it is very secure but slower.
- **Next-Generation Firewall (NGFW):** Combines stateful inspection with deep packet inspection, intrusion prevention, application awareness, and malware filtering.
- **Web Application Firewall (WAF):** Protects web applications from attacks such as SQL injection and cross-site scripting.

By deployment

- **Hardware firewall:** A dedicated device placed at the network edge.
- **Software (host-based) firewall:** Runs on an individual computer or server.
- **Cloud firewall:** Provided as a service in the cloud (firewall-as-a-service).

Procedure:

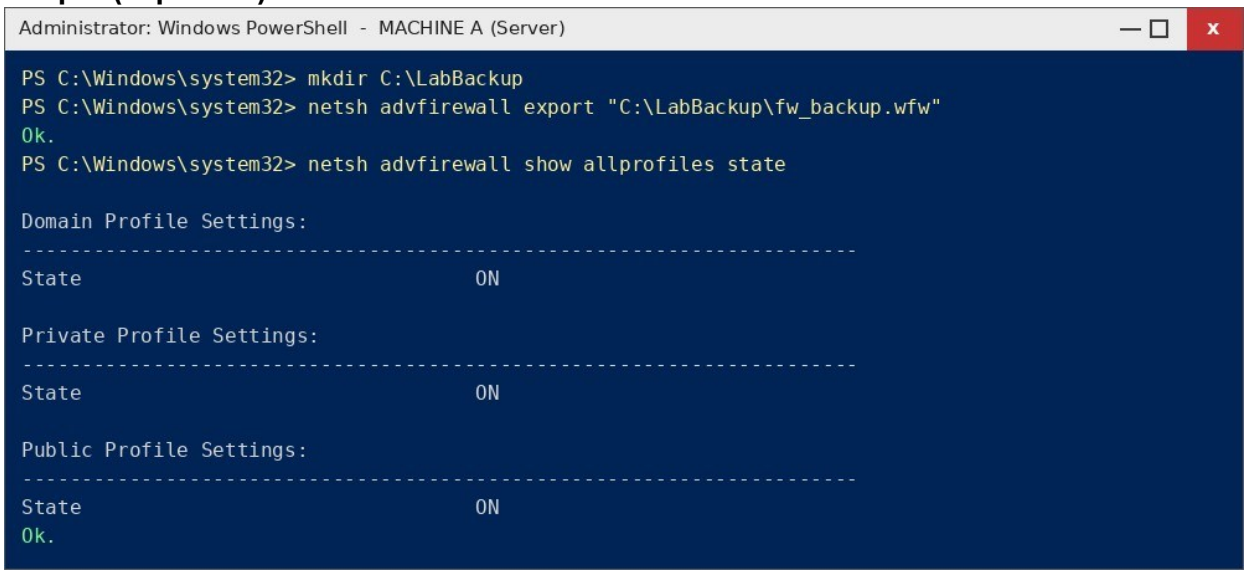
Step 1: Back up the firewall policy and check its status

Purpose: Save the current policy so it can be restored, and confirm the firewall is ON for all profiles.

Input (type this):

mkdir C:\LabBackup netsh advfirewall export "C:\LabBackup\fw_backup.wfw" netsh advfirewall show allprofiles state

Output (expected):



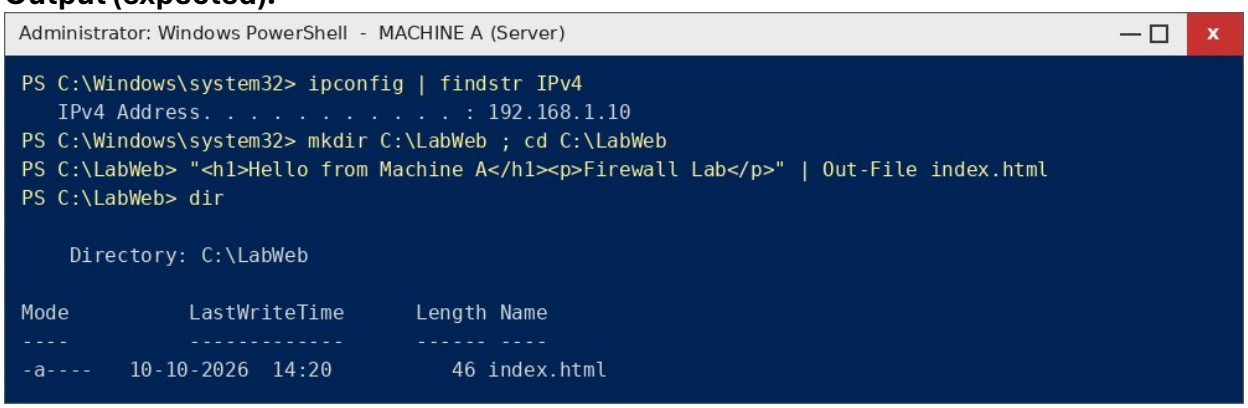
Step 2: Find the IP address and create the test web page

Purpose: Know the address that Machine B will test, and prepare a small web page to serve.

Input (type this):

ipconfig | findstr IPv4 mkdir C:\LabWeb ; cd C:\LabWeb "<h1>Hello from Machine A</h1> Firewall Lab " | Out-File index.html dir

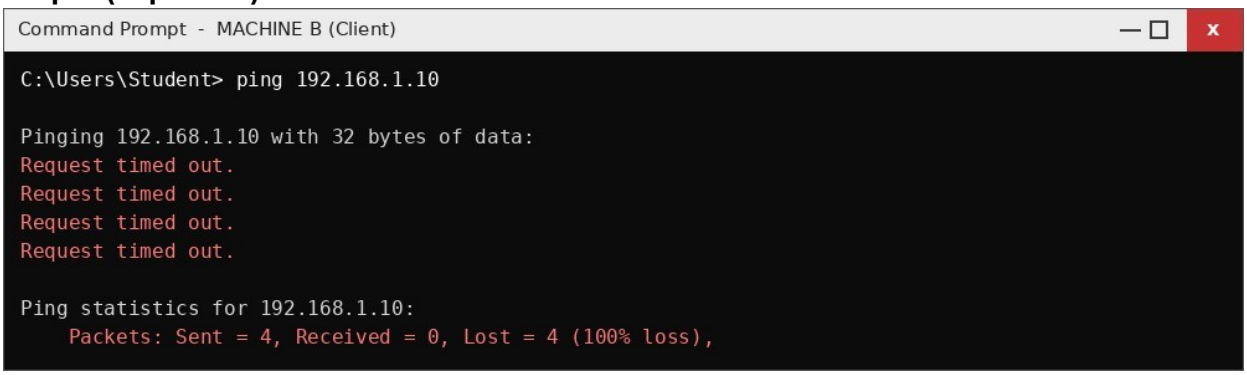
Output (expected):



Step 3: Baseline test: ping is blocked by default
Purpose: See how the firewall behaves before any rule is added.

Input (type this): ping 192.168.1.10

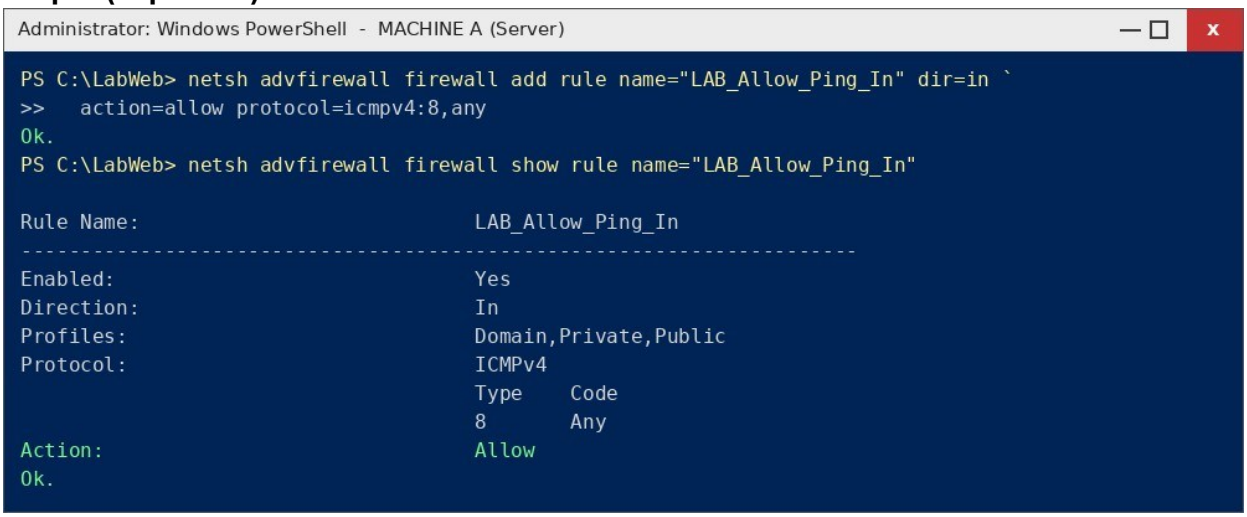
Output (expected):



Step 4: Rule 1: Allow ping (ICMP echo)
Purpose: Create an inbound allow rule for ICMPv4 echo requests (type 8) and verify it.

Input (type this):
netsh advfirewall firewall add rule name="LAB_Allow_Ping_In" dir=in ` action=allow protocol=icmpv4:8,any
netsh advfirewall firewall show rule name="LAB_Allow_Ping_In"

Output (expected):

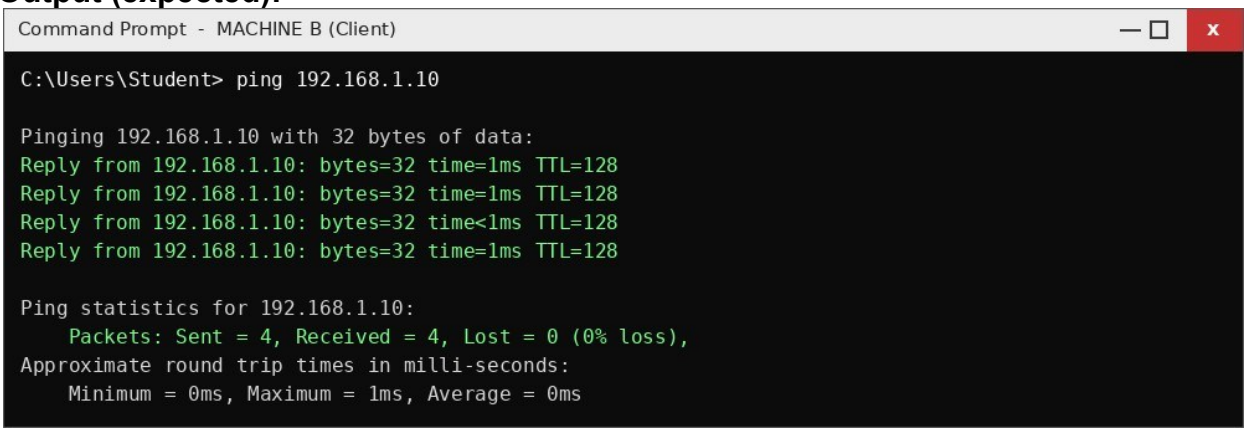


Step 5: Test Rule 1 from Machine B

Purpose: Confirm that the allow rule changed the result.

Input (type this): `ping 192.168.1.10`

Output (expected):



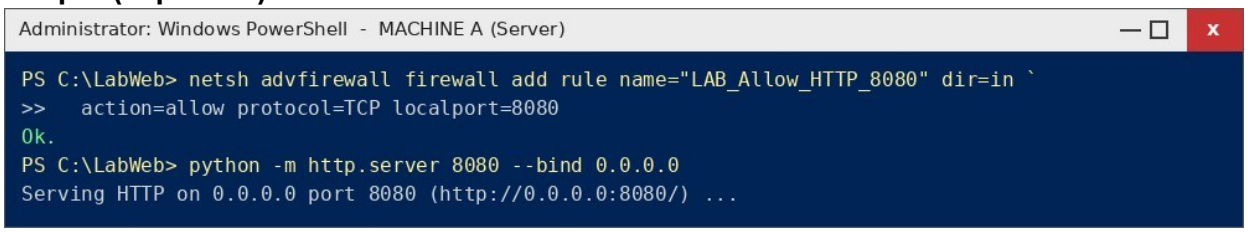
Step 6: Rule 2: Allow TCP port 8080 and start the web server

Purpose: Allow inbound web traffic on one specific port and run a simple service on that port.

Input (type this):

```
netsh advfirewall firewall add rule name="LAB_Allow_HTTP_8080" dir=in `
action=allow protocol=TCP localport=8080 cd C:\LabWeb
python -m http.server 8080 --bind 0.0.0.0
```

Output (expected):



Step 7: Test Rule 2 from Machine B

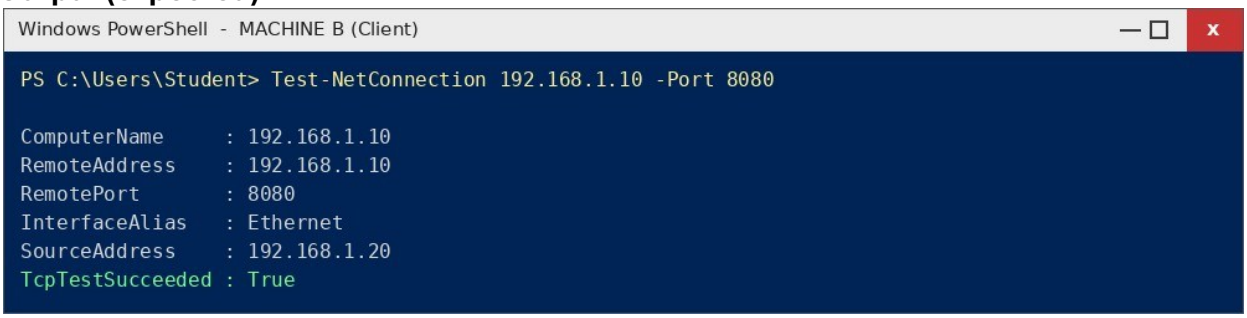
Purpose: Check that TCP port 8080 is now reachable, using both a command and a browser.

Input (type this):

Test-NetConnection 192.168.1.10 -Port 8080

then open in the browser: http://192.168.1.10:8080/

Output (expected):







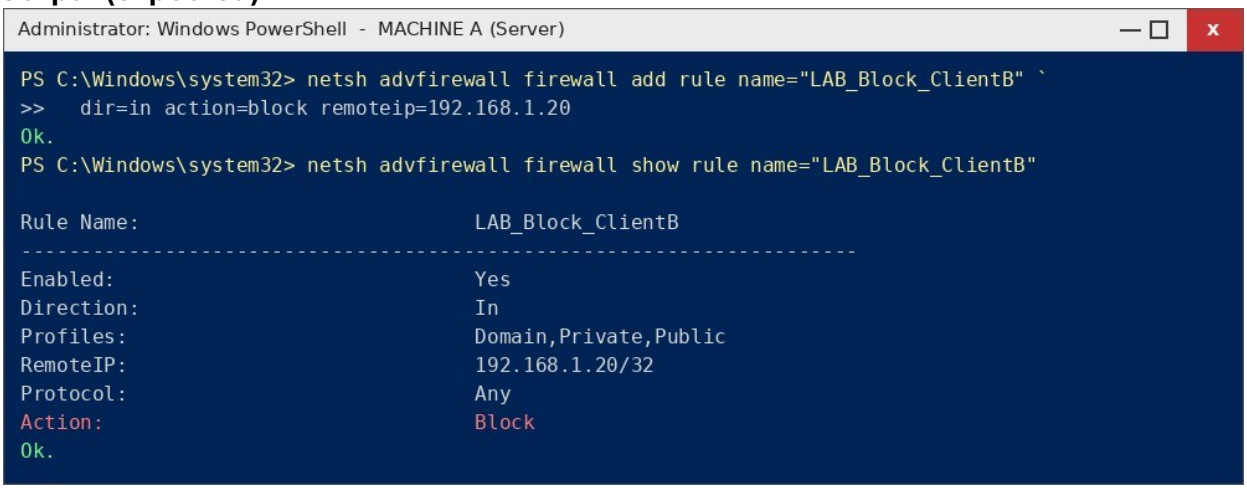
Step 8: Rule 3: Block one specific host (rule priority)

Purpose: Show that a Block rule overrides the Allow rules and can isolate a single IP address.

Input (type this):

netsh advfirewall firewall add rule name="LAB_Block_ClientB" ` dir=in action=block remoteip=192.168.1.20 netsh advfirewall firewall show rule name="LAB_Block_ClientB"

Output (expected):



Step 9: Test Rule 3 from Machine B

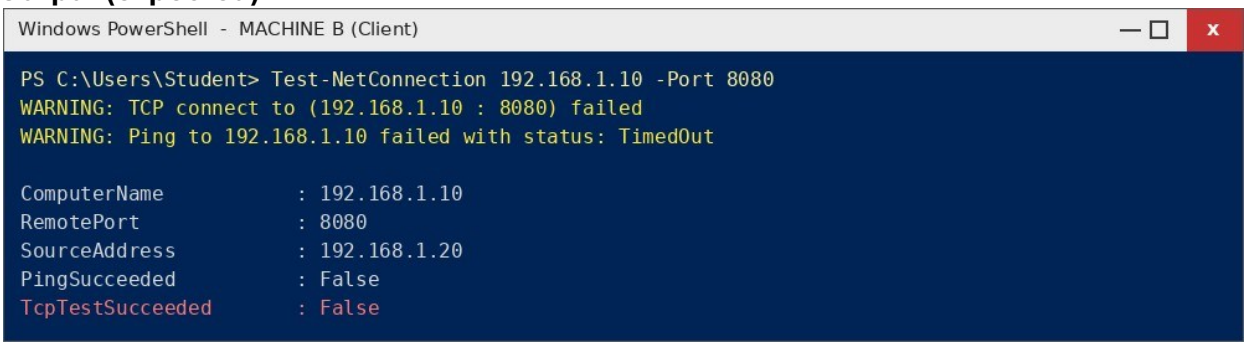
Purpose: Verify that both ping and the web port are now blocked.

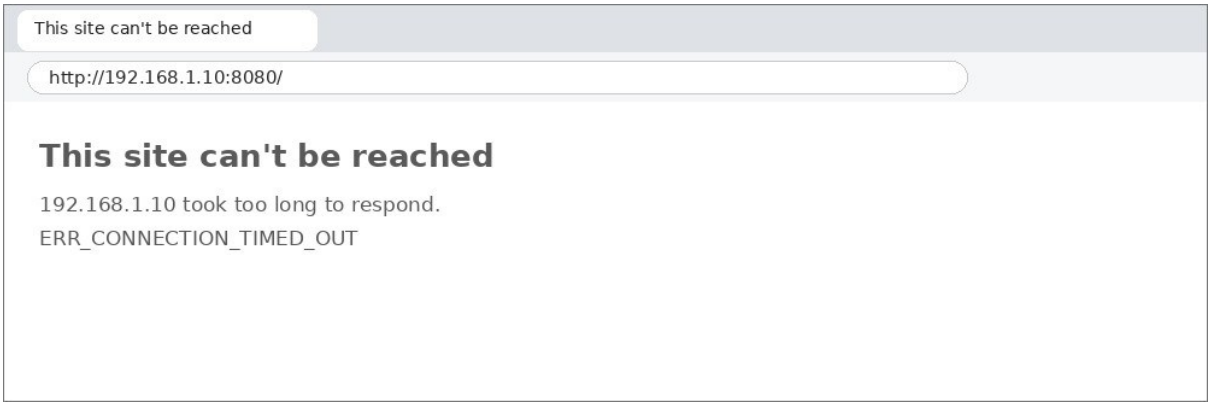
Input (type this):

Test-NetConnection 192.168.1.10 -Port 8080

reload http://192.168.1.10:8080/ in the browser

Output (expected):



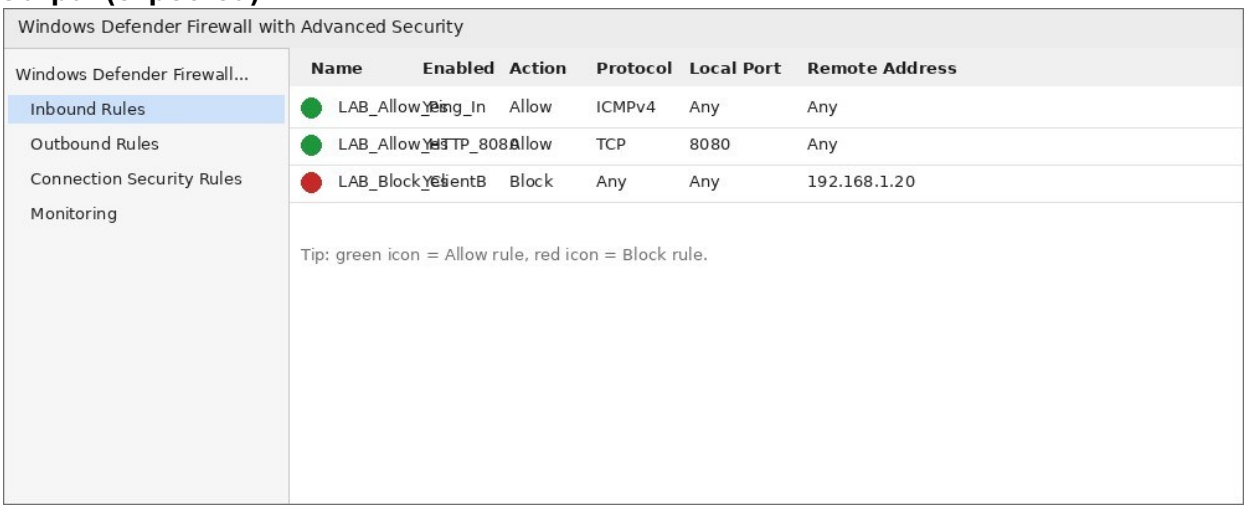


Step 10: View the rules in the graphical console

Purpose: See the same rules in the Windows Defender Firewall with Advanced Security console.

Input (type this): wf.msc (press Win + R, type wf.msc, press Enter; then click "Inbound Rules")

Output (expected):



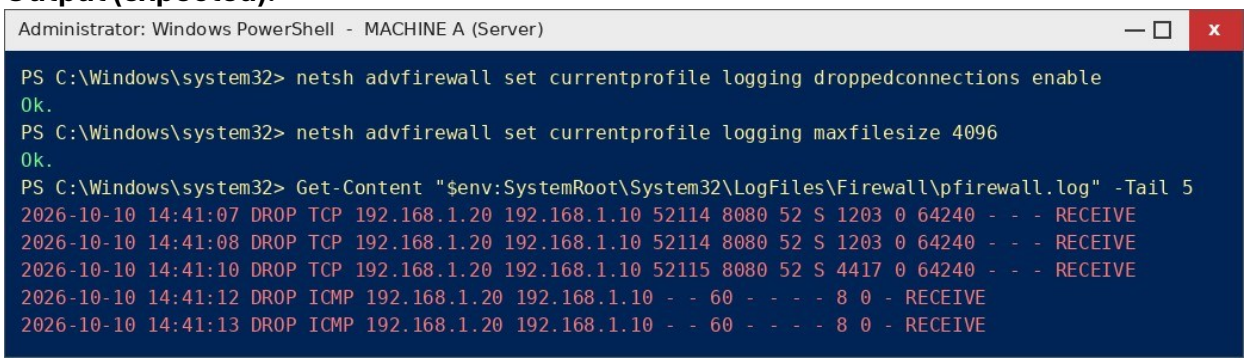
Step 11: Enable logging and read dropped packets

Purpose: Prove from the firewall log that the packets from Machine B were dropped.

Input (type this):

netsh advfirewall set currentprofile logging droppedconnections enable netsh advfirewall set currentprofile logging maxfilesize 4096 # On Machine B run the ping / Test-NetConnection tests again, then on A:
Get-Content "$env:SystemRoot\System32\LogFiles\Firewall\pfirewall.log" -Tail 5

Output (expected):



Step 12: Cleanup and restore

Purpose: Remove all lab rules, turn logging off and confirm the firewall is still ON.

Input (type this):

```
# Stop the web server first (press Ctrl + C in its window) netsh
advfirewall firewall delete rule name="LAB_Block_ClientB" netsh
advfirewall firewall delete rule name="LAB_Allow_HTTP_8080" netsh
advfirewall firewall delete rule name="LAB_Allow_Ping_In" netsh
advfirewall firewall show rule name=all | findstr "LAB_" netsh
advfirewall set allprofiles logging droppedconnections disable
netsh advfirewall show allprofiles state | findstr State # On
Machine B:
netsh advfirewall firewall delete rule name="LAB_Block_Out_8080"
```

Output (expected):

